

Intelligent Compliance Management System (ICMS)



MUHAMMAD ALI BUKHARI
01-135222-096

MUHAMMAD HAMMAD CHAGHTAI
01-135222-014

Supervisor: Dr. Faisal Bashir

Bachelor of Science in Information Technology

Department of Computer Science
Bahria University, Islamabad

April 2026

Certificate

We accept the work contained in the report title “**Intelligent Compliance Management System (ICMS)**” written by **Mr. Muhammad Ali Bukhari** and **Mr. Muhammad Hammad Chaghtai** as a confirmation to the required standard for the partial fulfillment of the degree of Bachelor of Science in Information Technology.

Approved by ...:

Supervisor: Dr. Faisal Bashir (Dean Engineering Sciences)

Internal Examiner: Name of the Internal Examiner (Title)

External Examiner: Name of the External Examiner (Title)

Project Coordinator: Dr. Kabeer Ahmed Bhatti (FYP Coordinator, IT Department)

Head of the Department: Dr. Muhammad Khurram Ehsan (Head of Department, Computer Science)

April , 2026

Abstract

Manual security auditing and static compliance mapping create blind spots that leave enterprise infrastructure vulnerable to configuration drift. Traditional Governance, Risk, and Compliance (GRC) processes often do not account for real-time endpoint telemetry, leading to high administrative overhead and inaccurate security assessments. This project introduces the Intelligent Compliance Management System (ICMS), a centralized platform that automates continuous controls monitoring. Developed as an industrial collaboration, the platform integrates with the Wazuh Security Information and Event Management (SIEM) system to ingest Security Configuration Assessment (SCA) logs. A deterministic mapping algorithm translates raw technical events into verifiable regulatory controls, which allows for continuous compliance tracking against standards like ISO/IEC 27001.

The system uses an N-tier architecture. A React Single Page Application (SPA) handles the presentation layer and communicates with a Django REST Framework API. To prevent main-thread blocking during SIEM polling and PDF report generation, the system offloads intensive I/O operations to an asynchronous Celery and Redis worker tier. Data persistence is managed via PostgreSQL, using a hybrid schema that combines relational constraints for Role-Based Access Control (RBAC) with JSON fields for telemetry storage. Platform security relies on stateless JSON Web Tokens (JWT) and Time-based One-Time Password (TOTP) two-factor authentication.

Testing confirmed the platform works as intended. The asynchronous tier processed high-density endpoint synchronizations with an average API latency under 150ms. The mapping engine maintained data integrity even during collision attempts. The ICMS shows that replacing manual audits with continuous SIEM integration establishes a verifiable compliance posture and reduces the burden of organizational audits.

Acknowledgments

In the name of Allah, the Most Gracious, the Most Merciful. All praise is due to Allah for the strength and guidance to complete this work.

We thank our supervisor, Dr. Faisal Bashir, for his mentorship and technical guidance. His insights into systems architecture and cybersecurity were essential to this project. We also appreciate the lab staff who assisted us and the faculty members who taught us over the last four years. We are thankful to the Department of Computer Science and Bahria University for providing the resources and environment needed to complete our degree.

We are grateful to our parents, families, and friends for their support and patience. We also thank each other for the teamwork required to handle the technical challenges of this project from start to finish.

Finally, on a personal note, I (Muhammad Ali Bukhari) dedicate this to my pet bird, Kiroo. Though he is no longer with me, his memory brought me comfort and joy throughout this journey.

Muhammad Ali Bukhari

Muhammad Hammad Chaghtai

Contents

Certificate	i
Abstract	ii
Acknowledgments	iii
List of Figures	x
List of Tables	xii
List of Abbreviations	xiv
1 Introduction	1
1.1 Project Background	1
1.1.1 The Temporal Blind Spot of Audits	1
1.1.2 The Shift to Continuous Monitoring	1
1.2 Problem Description	2
1.2.1 Telemetry Correlation Delays	2
1.2.2 Algorithmic Failures in Legacy Systems	2
1.3 Project Objectives	3
1.4 Methodology	4
1.4.1 Agile Development Lifecycle	4
1.4.2 Sandbox Testing and Evaluation	4
1.5 Project Scope	4
1.5.1 Compliance Framework Boundaries	4

1.5.2	Technical Implementation Boundaries	5
1.6	Feasibility Study	5
1.6.1	Technical and Economic Feasibility	5
1.6.2	Operational Feasibility	6
1.7	Areas of Application	6
1.7.1	Financial Technology (FinTech)	6
1.7.2	Healthcare Sector	6
1.8	Report Organization	6
2	Literature Review	8
2.1	Literature Review	8
2.1.1	Evolution of Governance, Risk, and Compliance (GRC) Platforms	8
2.1.2	Security Information and Event Management (SIEM) Integrations	8
2.1.3	Performance Bottlenecks in Telemetry Aggregation	9
2.1.4	Modern Access Control Paradigms	10
2.1.5	Containerization and Defense-in-Depth Topology	11
2.1.6	Cognitive Load and HCI in Security Dashboards	11
2.2	Analysis of Existing Systems	12
2.2.1	Manual Spreadsheet Auditing	12
2.2.2	Vanta (Commercial SaaS)	12
2.2.3	Drata (Commercial SaaS)	12
2.2.4	AuditBoard (Enterprise GRC)	13
2.3	Comparative Analysis	13
2.4	Summary of Advantages	14

3	Requirement Specification	16
3.1	Existing System	16
3.2	Proposed System	16
3.2.1	Technology Stack Selection	16
3.2.2	High-Level Architecture	17
3.2.3	Dockerized Infrastructure and Network Isolation	18
3.3	Requirements Specification	19
3.4	Functional Requirements	19
3.5	Non-Functional Requirements	20
3.6	User Requirements	21
3.7	System Models and Components	21
3.7.1	Context Diagram	21
3.7.2	Data Flow Diagram	22
3.8	Use Cases	23
3.8.1	Use Case Diagram	23
3.8.2	Descriptive Use Cases	24
4	Design	31
4.1	System Architecture	31
4.1.1	Client and Presentation Tier	31
4.1.2	Application and Middleware Tier	31
4.1.3	Asynchronous Processing Tier	31
4.1.4	Data Persistence Tier	32
4.2	Design Constraints	32

4.2.1	Technical Constraints	32
4.2.2	Infrastructure and Operational Constraints	33
4.3	Design Methodology	33
4.4	High Level Design	34
4.4.1	Component Diagram	34
4.4.2	Deployment Diagram	35
4.4.3	System Activity Diagrams	35
4.4.4	System Sequence Diagrams	41
4.5	Low Level Design	44
4.5.1	UML Class Diagram	44
4.6	Database Design	45
4.6.1	Physical Data Model (Core ERD)	45
4.6.2	Extended Entity Relationship Diagram (EERD)	46
4.7	Graphical User Interface (GUI) Design	47
4.7.1	Authentication and Identity Management	47
4.7.2	Superadmin Configuration Portal	48
4.7.3	Admin and Auditor Workspaces	49
4.7.4	Automated PDF Audit Artifacts	51
4.8	External Interfaces	52
4.8.1	Wazuh SIEM REST API	52
4.8.2	SMTP Notification Gateway	52
4.8.3	Outbound Webhook Integration	53
4.8.4	Interface Specification Matrix	53

5	Implementation	54
5.1	System Architecture	54
5.1.1	Presentation Layer Implementation (Frontend)	56
5.1.2	Middleware and API Gateway (Backend)	56
5.1.3	Asynchronous Processing Engine	57
5.1.4	Data Persistence Layer	57
5.1.5	Architectural Technology Stack	58
5.2	System Internal Components	59
5.2.1	Identity and Access Management (IAM) Module	60
5.2.2	Governance and Compliance Data Engine	60
5.2.3	SIEM Telemetry and Synchronization Module	61
5.2.4	Automated Audit and Reporting Module	61
5.2.5	Frontend State and Visualization Layer	61
5.3	Tools and Technologies	62
6	System Testing and Evaluation	64
6.1	Critical Appraisal and Limitations	64
6.2	Testing Environment and Methodology	64
6.3	Functional System Test Cases	65
6.4	Non-Functional System Testing	79
6.4.1	Performance and Load Testing	80
6.4.2	Security and Penetration Testing	80
6.4.3	Usability and Accessibility Testing	81
6.4.4	Compatibility and Deployment Testing	82

7	Conclusions	84
A	User Manual	85
A.1	Purpose	85
A.2	System Requirements	85
A.3	System Access	85
A.4	Main User Steps	86
A.5	Logout	87
A.6	Support	87
A.7	Note	88
	References	89

List of Figures

3.1	Refined N-Tier System Architecture and Protocol Flow	18
3.2	Docker Service Mesh: Port Exposure and Health-Check Dependencies . .	19
3.3	Context Diagram: High-Level System Interactions	22
3.4	Level-1 Data Flow Diagram detailing internal compliance processes. . . .	22
3.5	Advanced UML Use Case Diagram illustrating RBAC inheritance hierarchies, core compliance interactions, system boundaries, and integration with secondary external actors (Authenticator, SIEM, SMTP).	24
4.1	Tier-wise System Architecture of the ICMS Platform	32
4.2	Conceptual View: UML Component Diagram	34
4.3	Physical View: UML Deployment Diagram	35
4.4	Activity Diagram: Authentication and IAM Routing	36
4.5	Activity Diagram: Parallel Asynchronous Telemetry Ingestion	37
4.6	Activity Diagram: Iterative Compliance Evaluation and Worst-Case Priority	38
4.7	Activity Diagram: Automated PDF Audit Report Generation	39
4.8	Activity Diagram: Framework Rule Mapping and API Validation	40
4.9	Activity Diagram: Real-Time Alerting and Notification Fan-out	41
4.10	Sequence Diagram: Authentication Flow with UML Alt Fragment	42
4.11	Sequence Diagram: Background Telemetry Synchronization with UML Loop	43
4.12	Sequence Diagram: Framework Rule Mapping	43
4.13	Sequence Diagram: Report Generation with Alternative Data States	44
4.14	UML Class Diagram: Core Object-Relational Models	45

4.15	Physical Database Schema: Core Relational Data Model with Exact Data Types	46
4.16	Extended Entity Relationship Diagram (EERD) with Constraints and Attributes	47
4.17	GUI Design: Secure User Authentication Portal	48
4.18	GUI Design: Superadmin Global Compliance Dashboard	48
4.19	GUI Design: Framework and Wazuh Rule Mapping Interface	49
4.20	GUI Design: Monitored Endpoints and Telemetry Synchronization Status	49
4.21	GUI Design: System Admin Operational Dashboard	50
4.22	GUI Design: Security Auditor Read-Only Compliance View	50
4.23	GUI Design: Auditor Report Generation Modal	50
4.24	System Output: Automated PDF Report - Cover and Executive Summary	51
4.25	System Output: Automated PDF Report - Aggregated Framework Control Matrix	51
4.26	System Output: Automated PDF Report - Technical Wazuh Telemetry Evidence	52
5.1	Implemented N-Tier System Architecture	55
5.2	Implemented Service Topology and Container Networking	55

List of Tables

2.1	Comparative Feature Matrix: Analyzing GRC Platforms	14
3.1	Use Case - Authentication	25
3.2	Use Case - Manage Frameworks	26
3.3	Use Case - Map Controls	27
3.4	Use Case - Sync Telemetry	28
3.5	Use Case - View Dashboard	29
3.6	Use Case - Export Audit Reports	30
4.1	External Interface Technical Specifications	53
5.1	Architectural Technology Stack and Implementation Roles	59
5.2	Summary of Tools, Technologies, and Development Stack	62
6.1	TC-01: Valid Superadmin Authentication (Positive)	66
6.2	TC-02: Invalid Credential Rejection (Negative)	67
6.3	TC-03: Global Theme Toggling — Light/Dark Mode (Positive)	68
6.4	TC-04: UI RBAC View Segregation (Positive)	69
6.5	TC-05: Unauthorized API Write Attempt (Negative)	70
6.6	TC-06: Deterministic Framework Mapping (Positive)	71
6.7	TC-07: Duplicate Mapping Collision (Negative)	72
6.8	TC-08: Wazuh API Handshake and Token Retrieval (Positive)	73
6.9	TC-09: Wazuh Connection Timeout Exception (Negative)	74

6.10	TC-10: Asynchronous Telemetry Polling via Celery Beat (Positive) . . .	75
6.11	TC-11: Unstructured JSON Log Ingestion (Positive)	76
6.12	TC-12: Dynamic Compliance Score Recalculation (Positive)	77
6.13	TC-13: Comprehensive PDF Audit Report Generation (Positive)	78
6.14	TC-14: Asynchronous SMTP Alert Dispatch (Positive)	79
6.15	Performance Test Results	80
6.16	Security Test Results	81
6.17	Usability Test Task Completion Results	82
6.18	Compatibility and Deployment Test Results	83

List of Abbreviations

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
CCM	Continuous Controls Monitoring
CI/CD	Continuous Integration and Continuous Deployment
CPU	Central Processing Unit
DOM	Document Object Model
DRF	Django REST Framework
ERD	Entity-Relationship Diagram
FedRAMP	Federal Risk and Authorization Management Program
FISMA	Federal Information Security Modernization Act
FK	Foreign Key
GRC	Governance, Risk, and Compliance
GUI	Graphical User Interface
HCI	Human-Computer Interaction
HIPAA	Health Insurance Portability and Accountability Act
HMAC	Hash-based Message Authentication Code
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
I/O	Input / Output
ICMS	Intelligent Compliance Management System
IDE	Integrated Development Environment
IEC	International Electrotechnical Commission
ISMS	Information Security Management System
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions

NIST	National Institute of Standards and Technology
ORM	Object-Relational Mapping
OS	Operating System
PDF	Portable Document Format
PHI	Protected Health Information
PK	Primary Key
QA	Quality Assurance
RBAC	Role-Based Access Control
REST	Representational State Transfer
SaaS	Software as a Service
SCA	Security Configuration Assessment
SHA	Secure Hash Algorithm
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
SQL	Structured Query Language
SSL	Secure Sockets Layer
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TOTP	Time-based One-Time Password
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VPN	Virtual Private Network
WCAG	Web Content Accessibility Guidelines
XSS	Cross-Site Scripting

Chapter 1

Introduction

1.1 Project Background

1.1.1 The Temporal Blind Spot of Audits

Historical security auditing methodologies rely on manual evidence collection and point-in-time assessments. Organizations compile spreadsheets and static configuration snapshots to satisfy regulatory requirements. This approach creates temporal blind spots because the security posture remains unverified between audit cycles. Configuration drift occurs frequently after successful audits, leaving infrastructure vulnerable to exploitation despite maintaining formal compliance status.

1.1.2 The Shift to Continuous Monitoring

The cybersecurity industry addresses these vulnerabilities by moving toward Continuous Controls Monitoring (CCM). This shift replaces manual sampling with automated validation of security controls [1]. CCM platforms integrate with the technical environment to evaluate configurations, access logs, and system integrity. This automation ensures compliance reflects the current state of the environment instead of a historical record. Modern regulatory mandates require continuous visibility. NIST Special Publication 800-53 Revision 5 calls for a transition from static control assessments to ongoing monitoring to validate system integrity [2]. Similarly, the ISO/IEC 27001:2022 standard requires organizations to establish an Information Security Management System (ISMS) that evaluates risks continuously [3]. These frameworks require tools that map raw system data to governance objectives. Security Information and Event Management (SIEM) systems provide the data for this architecture. SIEM engines ingest telemetry from the enterprise and translate system events into verifiable compliance states. By establishing baselines, organizations move from reactive reporting to data-driven security management. This project implements a system that bridges the gap between technical SIEM

telemetry and regulatory control requirements.

1.2 Problem Description

1.2.1 Telemetry Correlation Delays

Translating large volumes of endpoint telemetry into regulatory frameworks is an operational limitation. SIEM platforms generate technical alerts, such as SSH authentication failures or unauthorized file modifications, which must be correlated against ISO 27001 or SOC 2 control matrices. Manual correlation processes introduce delays between event detection and compliance validation. This creates risk windows where system vulnerabilities remain unaddressed. Concurrent telemetry sources can produce contradictory states, which requires resolution mechanisms that manual workflows cannot provide.

1.2.2 Algorithmic Failures in Legacy Systems

Calculating compliance metrics across distributed fleets exposes data aggregation limits in traditional GRC platforms. Standard relational database implementations often suffer from the N+1 query problem, where the application executes sequential retrieval loops for individual security controls. This architectural flaw imposes a linear $O(N)$ algorithmic complexity on data retrieval, which causes performance to drop as the number of monitored agents increases. High latency prevents continuous monitoring because dashboard rendering times exceed thresholds for operational visibility. The algorithmic limitations of legacy GRC systems stem from their inability to process large datasets quickly. When an organization maps raw telemetry events to multiple regulatory frameworks, application-level iteration often exhausts memory bandwidth. By fetching full relational objects through unoptimized query loops, these systems saturate computational resources, which leads to timeouts and data inconsistency. This performance decay prevents the transition from point-in-time auditing to continuous monitoring because system latency outpaces telemetry ingestion.

Legacy GRC interfaces increase the cognitive load on security analysts. Rigid dashboards and unoptimized color schemes lead to alert fatigue. Empirical measurement of alerts in a live Security Operations Center found that a significant proportion of triggered alerts correspond to benign activity, which forces analysts to process excessive noise during incident triage [4]. This degrades incident response times and increases organizational

risk. High-density data grids lack a visual hierarchy, which forces operators to filter noise from critical signals manually. This structural rigidity induces alert fatigue, where the analyst's sensory threshold increases until critical exploits are overlooked. The absence of adaptive layout modules makes these tools a source of risk during security incidents.

1.3 Project Objectives

The ICMS platform addresses these technical and operational gaps through four engineering objectives:

- **Telemetry Automation:** Automate the ingestion and mapping of Wazuh SIEM telemetry to ISO 27001 and SOC 2 controls. The system uses a Worst-Case Priority algorithm to resolve conflicting evaluations. By polling the Wazuh Manager's Security Configuration Assessment (SCA) module, the platform retrieves check results and correlates them against a compliance matrix. This ensures the compliance dashboard reflects the most conservative security posture and minimizes false-positive reports.
- **Performance Optimization:** Eliminate the $N+1$ database querying bottleneck by using $O(1)$ set-based aggregations through the Django ORM. The platform uses query annotation and conditional SQL filtering to keep dashboard latency below 200 milliseconds. The system projects SQL constraints and aggregations directly onto the relational layer instead of executing sequential loops. By computing fleet-wide compliance scores internally, the system reduces data transfer overhead and memory pressure on the application server.
- **Security Infrastructure:** Enforce infrastructure and session security through a Dockerized virtual bridge network that isolates the PostgreSQL data tier, combined with custom Django REST Framework Role-Based Access Control and stateless JWT rotation. The platform isolates the database within a private virtual network so the persistence tier remains unreachable from external networks. This isolation pairs with an Identity and Access Management (IAM) layer that uses stateless JSON Web Token (JWT) rotation and custom permission classes.
- **UI/UX Engineering:** Reduce alert fatigue by using a modern frontend that replaces rigid tables with flexible layouts. The interface allows data rows to wrap and scale based on viewport size. The platform uses depth and color scaling to distinguish interactive security cards from background telemetry, which structures raw data for decision support.

1.4 Methodology

1.4.1 Agile Development Lifecycle

The ICMS platform used an Agile software development lifecycle for iterative progression. This approach began with a literature review and structural analysis of the ISO/IEC 27001:2022 and SOC 2 Type II regulatory frameworks. By breaking down governance mandates into technical variables, the development team created a schema that maps operational telemetry to compliance objectives. Following this, the methodology emphasized rapid prototyping of API integration layers. The team evaluated the Wazuh Security Configuration Assessment (SCA) module's capabilities and designed client connectors to poll and serialize endpoint states without overloading the central SIEM infrastructure.

1.4.2 Sandbox Testing and Evaluation

Development sprints focused on the architectural design and implementation of the back-end system. The methodology prioritized performance optimization by benchmarking database aggregation strategies against synthetic telemetry datasets. This resulted in the use of $O(1)$ set-based query projections to eliminate relational bottlenecks. In the evaluation phases, the platform moved to a containerized deployment model using Docker virtual bridge networks. This isolated environment allowed the team to perform security validation, performance profiling, and stress testing against simulated threats and compliance drift scenarios. By testing the platform within a sandbox, the methodology ensured the ICMS was resilient before enterprise integration.

1.5 Project Scope

1.5.1 Compliance Framework Boundaries

The ICMS platform operates within the ISO/IEC 27001:2022 and SOC 2 Type II compliance frameworks. The scope includes the ingestion and mapping of endpoint security telemetry from the Wazuh Manager's Security Configuration Assessment (SCA) module. By using the Wazuh REST API, the system retrieves check results from active agents and correlates them to governance controls. This data mapping ensures the compliance state represents the technical environment's configuration. The system initiates authen-

ticated requests to the SCA module to retrieve payloads describing the pass or fail status of configuration checks on monitored agents. Each check ID is cross-referenced against the ICMS seed matrix, which maps technical identifiers to governance controls. This correlation translates technical events into regulatory violations.

1.5.2 Technical Implementation Boundaries

The implementation boundary restricts the system to a containerized Docker ecosystem. This environment uses a private virtual bridge network to facilitate communication between the PostgreSQL database, Redis message broker, Celery workers, and the Django-React application stack. This architecture ensures the data persistence tier remains inaccessible to external networks, which maintains a baseline for storing compliance telemetry and organizational policies. Several operational domains remain outside the project scope. The system does not perform active Incident Response (IR) actions, such as isolating compromised endpoints or terminating malicious processes. The platform does not conduct network packet sniffing or automated remediation of configuration drift. The system evaluates and reports on the compliance state but does not change endpoint configurations; infrastructure remediation remains the responsibility of IT operations teams.

1.6 Feasibility Study

1.6.1 Technical and Economic Feasibility

The ICMS platform is technically and economically feasible. The platform uses the open-source Wazuh SIEM ecosystem alongside the Django and React frameworks, which ensures architectural dependencies can handle telemetry streams. Containerizing the application stack using Docker ensures deployment across different hardware environments. By decoupling the application logic from the host operating system, the system guarantees execution and allows organizations to deploy the solution on existing infrastructure without buying specialized hardware. Economically, the platform reduces costs compared to proprietary Software-as-a-Service (SaaS) compliance solutions. By using an open-source technology stack, the system eliminates recurring licensing fees and vendor lock-in risks. Organizations can achieve continuous controls monitoring using existing open-source SIEM infrastructure, shifting the financial model from high operational expenditure (OpEx) to a maintenance profile.

1.6.2 Operational Feasibility

The interface and automated telemetry mapping reduce the labor hours required for evidence collection, which increases the efficiency of Security Operations Centers (SOC).

1.7 Areas of Application

1.7.1 Financial Technology (FinTech)

The ICMS platform provides continuous monitoring for industries with strict data privacy and governance mandates. In the Financial Technology (FinTech) sector, where transaction processing and infrastructure scaling outpace point-in-time auditing, the platform provides the assurance required by regulatory bodies. FinTech startups processing payment data can use the platform to maintain adherence to SOC 2 Type II and internal security policies, which mitigates the risk of financial penalties or loss of licenses due to configuration drift in cloud environments.

1.7.2 Healthcare Sector

The platform also applies to the Healthcare sector for organizations managing Protected Health Information (PHI). Healthcare organizations validate their infrastructure against frameworks like HIPAA and ISO 27001 to prevent data breaches. The platform automates the translation of technical vulnerabilities into compliance violations, allowing hospital networks to maintain a consistent posture across distributed endpoints. By providing data sovereignty within an isolated Docker environment, the platform ensures operational telemetry remains within the organization's private network, which supports entities operating under confidentiality requirements.

1.8 Report Organization

This report consists of seven chapters detailing the design and implementation of the ICMS platform. Chapter 2 evaluates literature regarding Governance, Risk, and Compliance (GRC) frameworks and Security Information and Event Management (SIEM) integration. Chapter 3 defines the functional and non-functional requirements and establishes

the system architecture. Chapter 4 provides the system design, using Unified Modeling Language (UML) diagrams to represent logical and physical data flows. Chapter 5 focuses on implementation, detailing backend algorithms and containerized orchestration. Chapter 6 outlines testing protocols and evaluates performance against accuracy and latency metrics. Chapter 7 summarizes the project's contributions and proposes directions for future work.

Chapter 2

Literature Review

2.1 Literature Review

2.1.1 Evolution of Governance, Risk, and Compliance (GRC) Platforms

Historical security auditing relied on point-in-time assessments, which left blind spots between evaluation periods. Organizations managed Governance, Risk, and Compliance (GRC) through manual evidence collection, and this often made the compliance state obsolete shortly after an assessment. Regulatory frameworks now require a transition toward continuous controls monitoring architectures. Continuous Controls Monitoring has emerged as a recognized discipline for addressing these audit gaps and maintaining an up-to-date compliance state [5]. Standards like NIST Special Publication 800-53 Revision 5 and ISO/IEC 27001:2022 mandate ongoing monitoring to validate control implementation [2, 3]. SIEM-driven compliance offers several operational advantages. Centralized log management supports audit accountability by preserving data confidentiality, integrity, and availability. Automated analysis establishes operational baselines, which helps forensic investigations during security incidents. Security administrators configure SIEM agents to track software inventories, hardware configurations, and system health. This data feeds into the GRC database to replace reactive audits with a data-driven security posture. By evaluating log streams against baselines, organizations turn evidence generation into a part of daily security operations.

2.1.2 Security Information and Event Management (SIEM) Integrations

The integration of Security Information and Event Management (SIEM) systems changes the GRC methodology. SIEM platforms are telemetry ingestion engines that replace manual log retrieval with automated data correlation. NIST SP 800-53 defines control

families, such as Audit and Accountability (AU) and System and Information Integrity (SI), that require automated tracking to satisfy continuous validation requirements [2]. Open-source SIEM solutions such as Wazuh have been empirically evaluated and shown to provide adequate security coverage for organisations operating under budget constraints [6]. Platforms like Wazuh, an open-source SIEM and Extended Detection and Response (XDR) solution, facilitate this by providing aggregation logic that maps compliance events against NIST and ISO matrices. Automated compliance mapping fills the gap between technical events and business risk. When a user modifies a protected configuration file, File Integrity Monitoring (FIM) sensors generate an event log. The SIEM correlates this event against framework matrices and flags the action as a violation of ISO 27001 Annex A.8.9 (Configuration Management) or NIST SP 800-53 control CM-6. Analysts monitor deviations through dashboards that aggregate alert volumes and isolate non-compliant agents for remediation. The Wazuh Security Configuration Assessment (SCA) module has been applied to automate CIS benchmark compliance, demonstrating its suitability as a continuous validation engine [7].

2.1.3 Performance Bottlenecks in Telemetry Aggregation

Calculating compliance metrics across distributed fleets exposes data aggregation limits in traditional GRC platforms. Standard relational database implementations often suffer from the N+1 query problem, where the application executes sequential retrieval loops for individual security controls. This architectural flaw imposes a linear $O(N)$ algorithmic complexity on data retrieval, which causes performance to drop as the number of monitored agents increases. High latency prevents continuous monitoring because dashboard rendering times exceed thresholds for operational visibility. The overhead of Object-Relational Mapping (ORM) frameworks extends beyond query volume. Application-level data processing, where the system retrieves full object graphs before filtering or aggregating in memory, creates bandwidth bottlenecks. This pattern forces the application server to create transient objects, which triggers garbage collection cycles that can block compliance logic. In environments where SIEM agents transmit telemetry continuously, these $O(N)$ operations lead to a loss of operational visibility.

Relational data access requires moving from sequential $O(N)$ loops to $O(1)$ set-based aggregations. Research into field flexibility in relational databases demonstrates that JSON-capable schemas provide the storage flexibility needed for variable telemetry data without sacrificing the query performance of the relational engine [8]. Developers optimize ORM queries by using eager loading to translate application logic into SQL JOIN clauses. Fetching related entities in a single query allows the database engine to use B-Tree indices

efficiently, which reduces execution time and minimizes context switching between the application runtime and the database server. Set-based database aggregations move the workload to the relational engine. Using query projection, the system instructs the database to compute sums, averages, and conditional compliance counts internally. This reduces data transfer across the network from megabytes of row data to simple integer values. By using conditional aggregation functions, the platform achieves a consistent $O(1)$ response time. These optimizations ensure dashboard rendering remains responsive as the organizational asset inventory grows.

2.1.4 Modern Access Control Paradigms

Securing the compliance data layer requires Identity and Access Management (IAM) protocols. Role-Based Access Control (RBAC) provides the foundation for authorization in distributed systems. RBAC methodologies control access to system resources based on roles assigned to a user instead of assigning permissions directly to a user identity [9]. Authentication in distributed microservices often relies on stateless session security, usually implemented via JSON Web Tokens (JWT). The authorization server signs the JWT using asymmetric encryption to prevent tampering and allow local verification without a database lookup. Authorization in the ICMS platform uses an enforcement layer that applies Zero Trust principles to the API routing logic. The system evaluates the identity context of every inbound request. Custom permission classes intercept traffic at the view layer and verify that the requester has the RBAC attributes required for the target resource. This enforcement is decoupled from the business logic, which ensures a consistent security boundary.

The security lifecycle of stateless JSON Web Tokens (JWT) includes rotation policies to mitigate risks from long-lived credentials. Traditional session IDs require database lookups for verification; JWTs carry signed claims that allow microservices to authenticate users locally. To prevent session hijacking, the platform uses a refresh token rotation scheme. When the client requests a new credential, the server invalidates the previous refresh token and issues a new pair. Research into TOTP-based cloud storage systems confirms that combining JWT tokens with time-based one-time passwords provides a strong multi-factor authentication baseline that defends against replay and session hijacking attacks [10]. The architecture also uses Time-Based One-Time Passwords (TOTP) as a secondary authentication factor. TOTP algorithms rely on a cryptographic secret shared between the server and the client's authenticator app. Both entities use the current Unix timestamp as a factor in a Keyed-Hash Message Authentication Code (HMAC-SHA1) function. Because the 6-digit numeric code changes every 30 seconds, intercepted creden-

tials have limited use. Studies on two-factor authentication architectures confirm that TOTP-based schemes introduce meaningful resistance against replay attacks when combined with short-lived JWT rotation [11]. Client-side interceptors monitor for specific HTTP response states; when the server returns a forbidden status due to a missing or invalid TOTP challenge, the application redirects the user to an authentication view. This lifecycle ensures compliance data remains inaccessible to compromised endpoints.

2.1.5 Containerization and Defense-in-Depth Topology

Enterprise architecture is moving from monolithic software design toward microservices and containerized deployments. Monolithic systems couple application logic, user interfaces, and database drivers into a single deployment artifact, where a failure in one component can affect the entire environment. Containerization technologies, specifically Docker, provide a standardized runtime environment for distributed microservices. Research into the state-of-the-art of container technologies confirms that Docker offers strong isolation primitives and orchestration benefits for multi-tier web applications [12]. This ensures scalability and bounded contexts through Linux kernel features. Deploying the ICMS components via isolated Docker virtual bridge networks establishes a defense-in-depth topology. The architecture separates the public-facing API gateway layers from the private persistent storage layers, such as PostgreSQL and Redis. By binding these data tiers to internal Docker bridge networks, the system remains inaccessible to external scans. This isolation prevents actors from establishing direct TCP connections with the database and forces interactions through the authenticated business-logic layer. Docker containers also enable reproducible builds, which ensures consistent execution environments across development and production [13].

2.1.6 Cognitive Load and HCI in Security Dashboards

Security Operations Centers process large volumes of diagnostic data, which can trigger alert fatigue. Legacy GRC platforms present analysts with unfiltered data in dense tables, leading to sensory overload and diagnostic errors. Decision support tooling for SOC analysts has been shown to improve signal prioritization and reduce analyst cognitive strain [14]. Research into the empirical distribution of SOC alerts demonstrates that a large fraction of triggered alerts correspond to benign events, confirming that unfiltered data presentation is operationally harmful [4]. Reducing cognitive strain requires applying User Interface (UI) design principles grounded in human-computer interaction research. The ICMS platform uses modular card components and color scaling, such as red for

critical exploits and grayscale for benign logs, to guide the analyst’s focus. Using depth distinctions separates interactive components from background data, which structures telemetry into a decision-support tool.

2.2 Analysis of Existing Systems

2.2.1 Manual Spreadsheet Auditing

Traditional governance relies on labor-intensive processes including spreadsheets, task-tracking tickets, and point-in-time audits. This fragmented approach fails to scale in modern IT environments because the compliance state captured during an audit becomes obsolete once the assessment ends. The reliance on manual sampling and evidence collection creates blind spots where vulnerabilities remain undetected between audit cycles. The structural limits of spreadsheet applications lead to performance issues and data corruption when tracking many configuration parameters. Manual mapping also increases the probability of human error as analysts interpret technical data against regulatory constraints.

2.2.2 Vanta (Commercial SaaS)

Vanta operates in the commercial Software-as-a-Service (SaaS) continuous monitoring sector. The platform speeds up evidence collection by deploying proprietary agents and using API connectors to integrate with cloud providers and identity systems. This commercial solution introduces risks regarding data sovereignty and vendor lock-in. As a multi-tenant cloud environment, Vanta requires organizations to transmit security telemetry to its external infrastructure. This data transmission increases the attack surface. Vanta is a proprietary system, so its closed-source algorithms prevent organizations from independently verifying how telemetry events map to compliance controls. This lack of transparency forces organizations to rely on the provider’s internal security assertions.

2.2.3 Drata (Commercial SaaS)

Drata provides automated compliance tracking through a multi-tenant SaaS architecture. While the platform offers integrations and pre-mapped framework tracking, it involves risks regarding external data storage. Organizations using Drata export their infrastruc-

ture metadata to the provider’s cloud, which can violate data sovereignty requirements in the defense and financial sectors. The closed-source nature of Drata restricts security engineers from inspecting the underlying evaluation logic. This prevents custom modifications to the compliance engine, meaning that organizations with unique architectures cannot extend the system without relying on the vendor. This ensures the platform remains a closed-box system rather than a verifiable tool.

2.2.4 AuditBoard (Enterprise GRC)

AuditBoard targets the enterprise GRC market and focuses on risk management, ESG reporting, and internal audit synchronization. Despite its features, AuditBoard can be rigid to integrate. The platform has high licensing costs and requires professional service engagements for deployment, resulting in long setup cycles. This enterprise architecture compares poorly against containerized solutions that deploy quickly. AuditBoard relies on proprietary connectors to interface with security tools, which establishes a closed-box mapping approach. Security engineers cannot manipulate the logic linking SIEM alerts to organizational risk matrices. This opacity limits the platform’s utility in environments that require verifiable compliance states.

2.3 Comparative Analysis

The ICMS platform is an alternative for organizations requiring data sovereignty, technical transparency, and verifiable logic. By integrating with the open-source Wazuh SIEM, the platform achieves automation without needing external data transmission. The architecture provides control via the Django framework, which allows organizations to audit, modify, and extend the compliance evaluation algorithms. The single-tenant Docker topology ensures that security telemetry remains within an isolated, customer-controlled bridge network, which reduces the attack surface associated with multi-tenant cloud environments.

As shown in Table 2.1, the ICMS addresses the performance issues observed in proprietary systems through database engineering. By implementing $O(1)$ set-based aggregations, the platform processes telemetry logs efficiently. This ensures the platform maintains lower latency compared to legacy systems with monolithic database scaling constraints.

Table 2.1: Comparative Feature Matrix: Analyzing GRC Platforms

Feature	Traditional Methods (Manual)	Commercial SaaS (Vanta / Drata / AuditBoard)	ICMS Platform
Deployment Model	Decentralized documents	Multi-tenant public cloud (SaaS risk)	Single-tenant, isolated Docker bridge
Data Sovereignty	High (Data remains local)	Low (Data transmitted to third parties)	High (Data confined to internal network)
Performance Complexity	Prone to spreadsheet limits and crashes	Proprietary ORM scaling constraints, API latency	$O(1)$ set-based aggregation via <code>.annotate()</code>
Customizability	Manual modification	Rigid black-box mapping algorithms	Full programmatic control via Django ORM
Cost	High operational labor expenditure	Expensive recurring enterprise licensing	Zero licensing cost (Open-source stack)

2.4 Summary of Advantages

The ICMS platform provides operational and technical advantages that address the flaws of legacy auditing and commercial SaaS offerings:

- **Data Sovereignty via Docker Bridge Isolation:** The platform confines all PostgreSQL database interactions to a private virtual network, which ensures compliance telemetry remains isolated from the public internet and external third-party vendors.
- **Transparent, Verifiable Mapping:** By using an open-source Django backend, organizations can programmatically inspect and verify the logic linking raw Wazuh SIEM alerts to governance controls.
- **Deterministic $O(1)$ Database Performance:** Set-based aggregations replace standard ORM loops, allowing the system to render compliance dashboards with low latency regardless of the agent inventory size.
- **Zero-Cost Licensing Architecture:** The operational stack relies on open-source technologies like Wazuh, PostgreSQL, and Python, which removes the financial burden of commercial SaaS fees.
- **Automated Telemetry Deduplication:** The implementation of a Worst-Case

Priority algorithm ensures that conflicting telemetry reports from different sensors are resolved consistently, which prevents false-positive compliance states.

Chapter 3

Requirement Specification

3.1 Existing System

Traditional compliance management systems use manual spreadsheets and fragmented Software-as-a-Service (SaaS) platforms. Governance teams often map regulatory standards, such as ISO/IEC 27001:2022, to technical controls using decentralized documents. This approach is prone to human error, lacks version control, and often fails to maintain consistency across different frameworks. Manual tracking of configuration states increases operational labor and administrative overhead.

Existing auditing solutions perform delayed security assessments, which means data from internal systems is often out of date by the time it is reviewed. Legacy platforms lack specialized relational constraints, which causes latency when processing telemetry and makes it difficult to resolve vulnerabilities consistently. The reliance on proprietary SaaS architectures also introduces data sovereignty risks, as compliance metrics and configurations are transmitted to public cloud environments.

3.2 Proposed System

The ICMS platform provides a centralized system to address the limitations of manual mapping and asynchronous SIEM aggregation. The system uses a multi-tier architecture to maintain a separation of concerns while keeping compliance tracking within an isolated environment.

3.2.1 Technology Stack Selection

The Django framework provides a backend environment with an integrated Object-Relational Mapper (ORM) and built-in security middleware. The Django ORM allows for performance optimizations like $O(1)$ set-based database aggregations. By using query

projections and conditional filtering, the system keeps high-volume telemetry manageable. The Django REST Framework (DRF) adds serialization pipelines and supports authentication methods like stateless JSON Web Token (JWT) rotation.

React 18 and Vite are used to build a Single Page Application (SPA) for data visualization. React's Virtual DOM allows the interface to update compliance states without blocking the browser thread. This ensures the dashboard remains responsive when viewing large amounts of SIEM alerts. Vite improves the development process by using native ES modules, which reduces build times compared to older tools.

PostgreSQL is the primary database and provides the ACID compliance needed for organizational records. The database handles concurrent read/write loads from SIEM telemetry and automated mapping. By using B-Tree indexing and relational constraints, PostgreSQL ensures compliance metrics remain accurate. This satisfies the requirements for ISO 27001 and SOC 2 audits, where the history of security controls must be verifiable.

To keep the platform responsive during data-heavy operations, the architecture uses Celery and Redis as an asynchronous layer. This decouples tasks like telemetry synchronization from the main application thread to prevent timeouts. Redis is the message broker that moves task payloads between the web server and the background workers. This model allows the platform to scale its ingestion capacity without slowing down the primary API service.

3.2.2 High-Level Architecture

The Presentation Tier is a React SPA that manages user sessions and provides a monitoring interface. It communicates with the API via JSON payloads over HTTPS. By handling state management and rendering on the client side, the system keeps server resources available for data processing and mapping.

The Application Logic Tier, powered by Django REST Framework, manages compliance tasks. This tier handles the Identity and Access Management (IAM) lifecycle and JWT rotation. The Django ORM manages interactions with the database to enforce integrity and execute query aggregations. Asynchronous tasks use Celery and a Redis-based message broker. The system uses a Pub/Sub model to enqueue synchronization tasks, which allows Celery workers to process telemetry from the external Wazuh SIEM Manager. Figure 3.1 shows the protocol flow and multi-tier layout.

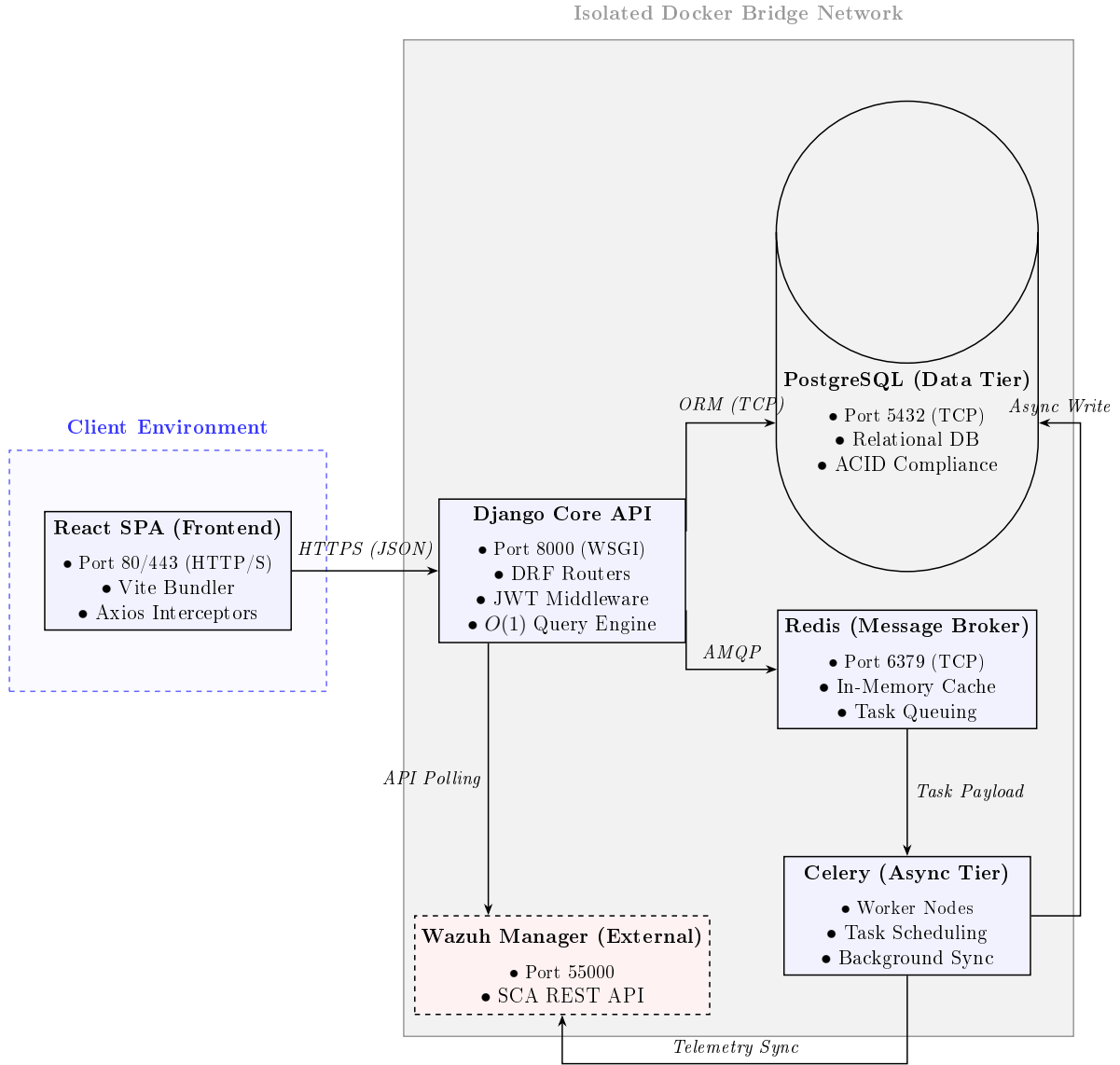


Figure 3.1: Refined N-Tier System Architecture and Protocol Flow

3.2.3 Dockerized Infrastructure and Network Isolation

A Docker virtual bridge network isolates the database from external environments. By keeping PostgreSQL and Redis on a private, internal network, the architecture ensures they are not accessible to external scans. The system uses Docker volumes to prevent data loss across container restarts. The startup sequence uses health-checks to ensure the platform is resilient during deployment. The configuration ensures the Django API container waits for other services before running migrations or starting the server. Figure 3.2 shows the isolation and health monitoring network.

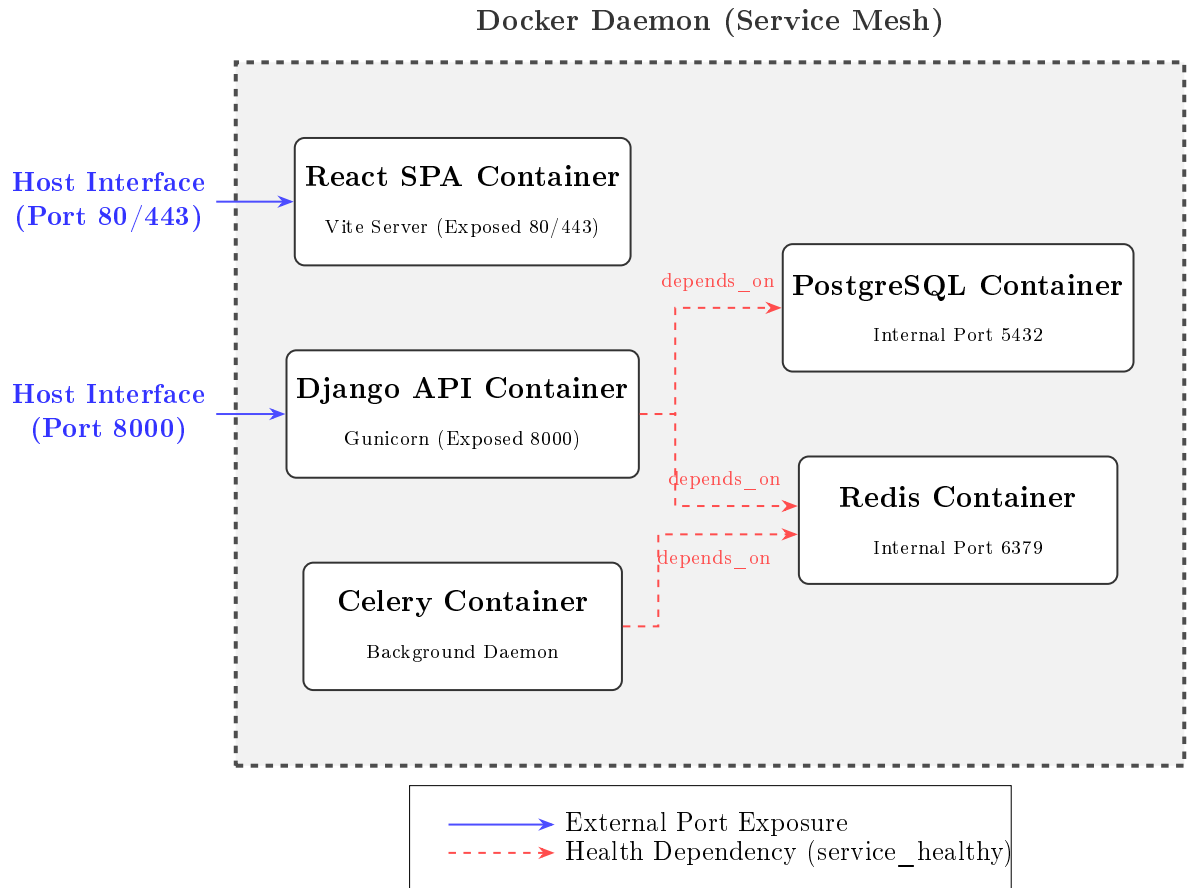


Figure 3.2: Docker Service Mesh: Port Exposure and Health-Check Dependencies

3.3 Requirements Specification

Requirement engineering for the ICMS platform involves mapping regulatory mandates to technical specifications. Aligning the platform with ISO/IEC 27001:2022 and SOC 2 Type II standards requires translating governance objectives into functional capabilities. This ensures architectural components satisfy compliance and operational constraints. The analysis phase establishes the foundation for a system that maintains visibility by prioritizing automated telemetry ingestion and data aggregation.

3.4 Functional Requirements

The functional requirements define the capabilities the ICMS architecture must have to operate:

- The system must use a 3-tier Role-Based Access Control (RBAC) model with Time-Based One-Time Password (TOTP) authentication.
- The platform must automate the ingestion of security telemetry from the Wazuh SCA module via a REST API.
- The system must resolve conflicting telemetry states using a Worst-Case Priority algorithm to ensure data integrity.
- The platform must generate compliance reports in PDF format for audit documentation.
- The system must provide an interface for managing frameworks, controls, and mapping identifiers.
- The application must use a stateless session lifecycle with JSON Web Token (JWT) rotation.

3.5 Non-Functional Requirements

The non-functional requirements set the operational constraints and performance thresholds for system reliability:

- The system must isolate PostgreSQL and Redis within a private Docker virtual bridge network.
- The backend must use $O(1)$ set-based query aggregations to keep dashboard latency below 200 milliseconds.
- The architecture must use asynchronous background tasks via Celery and Redis to keep the UI responsive.
- The frontend must use a responsive architecture to reduce analyst alert fatigue.
- The platform must ensure high availability through task retry logic and message queuing.
- The system must maintain an audit log that records administrative actions and system events.

3.6 User Requirements

The efficiency of the ICMS platform relies on a user experience tailored to its primary actors: the System Administrator and the Security Auditor. The System Administrator needs a control plane to manage infrastructure settings without directly accessing the database. They must be able to configure synchronization, manage RBAC privileges, and override automated telemetry when necessary. The interface must present these controls clearly, using visual feedback to confirm operations within the Django ORM.

The Security Auditor uses the platform to assess regulatory adherence. Their primary requirement is viewing compliance data without unnecessary clutter. The auditor needs an RBAC boundary that prevents them from modifying infrastructure mapping while providing access to historical logs and PDF exports. The React SPA architecture must ensure the dashboard remains responsive during telemetry ingestion by using techniques like memoization to prevent rendering delays.

3.7 System Models and Components

This section outlines the data exchange and flow used by the platform to synchronize SIEM telemetry with the internal compliance engine.

3.7.1 Context Diagram

The Context Diagram defines the interaction between the ICMS and external entities, showing the boundaries of the application and the data streams required. Figure [3.3](#) shows the flow from human operators to infrastructure components.

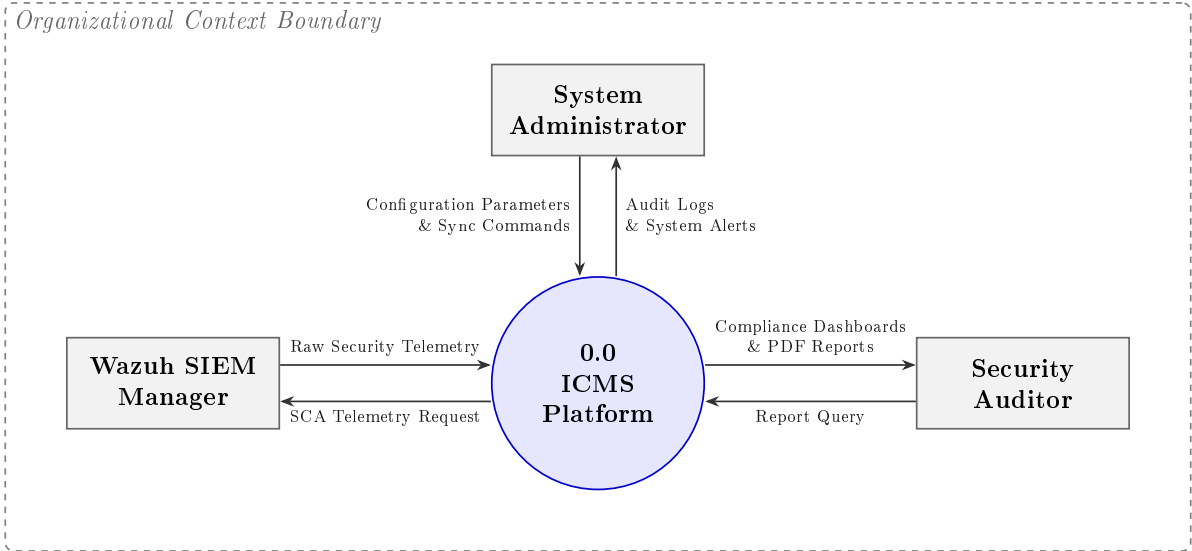


Figure 3.3: Context Diagram: High-Level System Interactions

3.7.2 Data Flow Diagram

The Level-1 Data Flow Diagram (DFD) shows how data payloads move through the platform's architecture and how raw SIEM logs become compliance metrics. Figure 3.4 outlines the pipeline used by the ICMS.

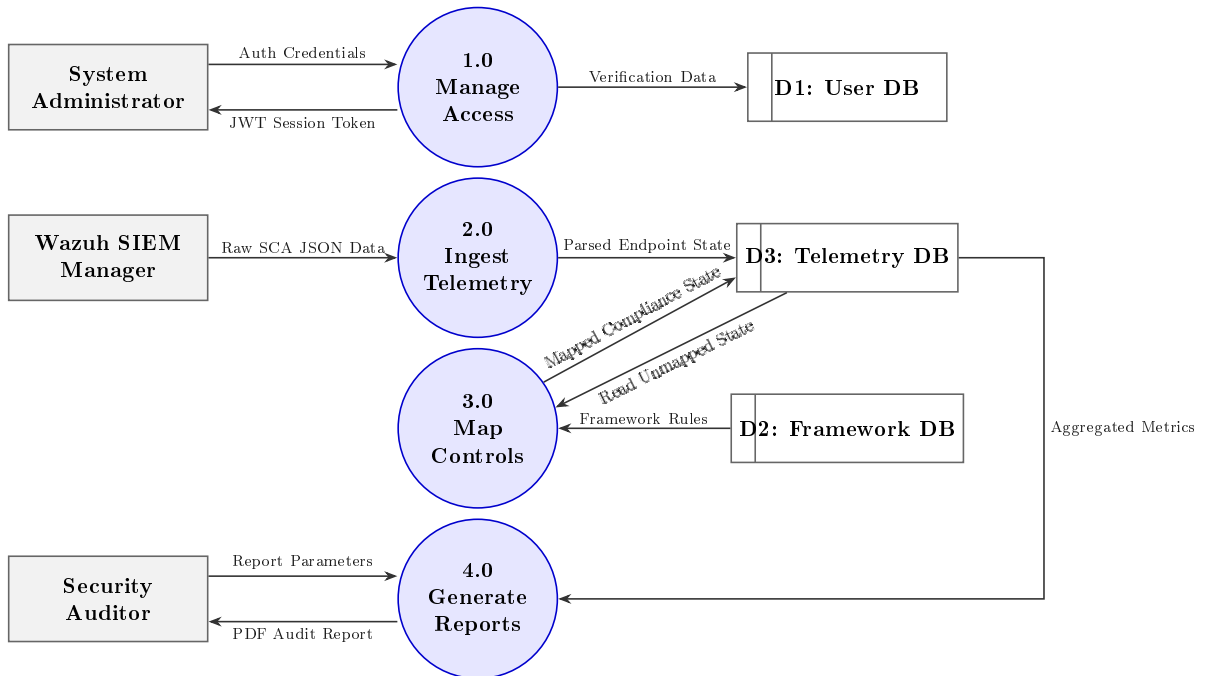


Figure 3.4: Level-1 Data Flow Diagram detailing internal compliance processes.

3.8 Use Cases

The use cases define the interaction sequences available to authenticated actors, providing deterministic state definitions for platform operations.

3.8.1 Use Case Diagram

Figure [3.5](#) shows the workflows for the System Administrator and the Security Auditor, establishing boundaries based on the platform's RBAC implementation.

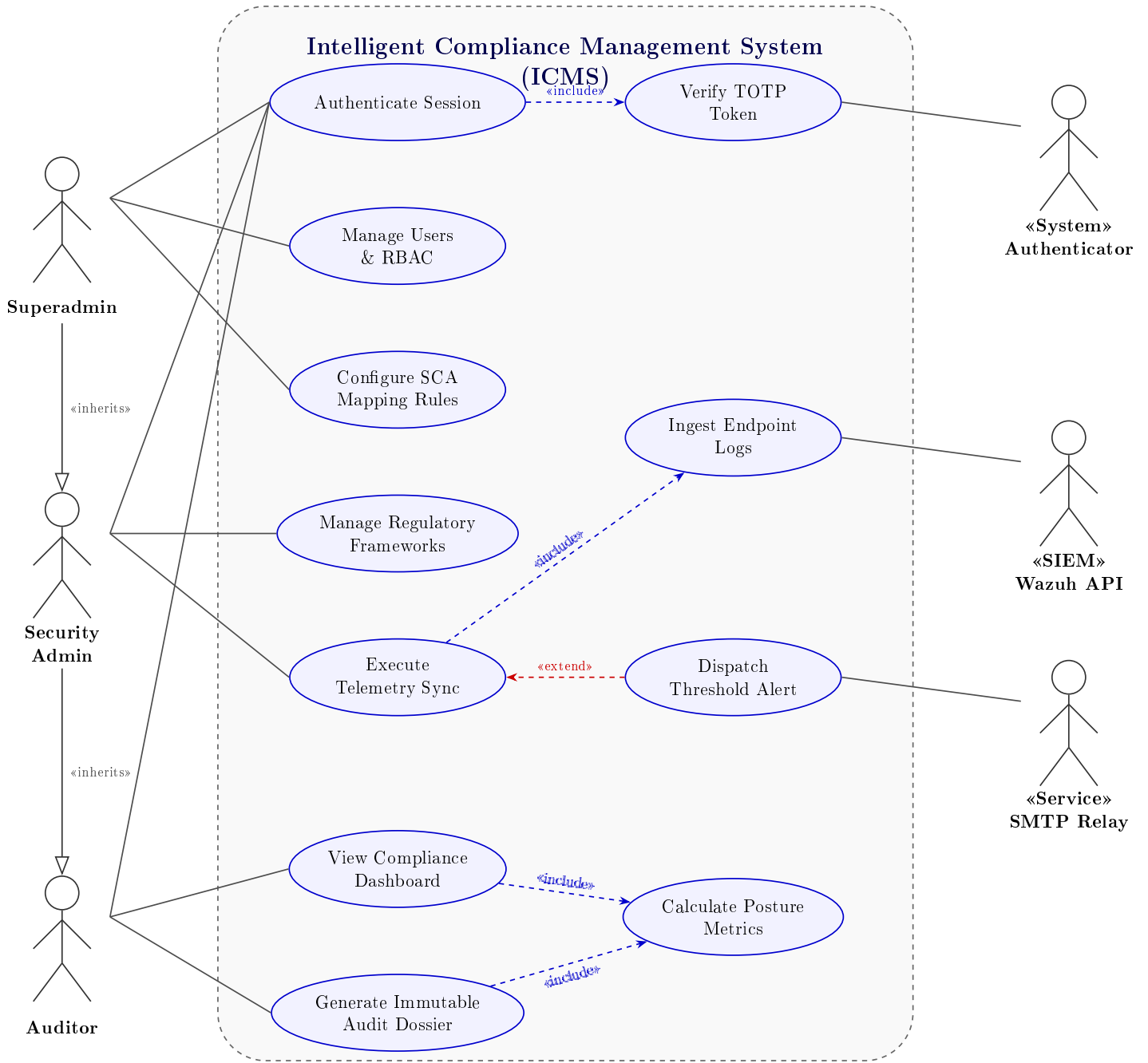


Figure 3.5: Advanced UML Use Case Diagram illustrating RBAC inheritance hierarchies, core compliance interactions, system boundaries, and integration with secondary external actors (Authenticator, SIEM, SMTP).

3.8.2 Descriptive Use Cases

The following tables define the flows, authentication requirements, and persistence states for each operation.

Table 3.1: Use Case - Authentication

Attribute	Description
Use Case ID	UC-01
Use Case Name	User Authentication
Primary Actor	System Administrator, Security Auditor
Preconditions	The user account is registered and active in the system.
Main Flow	1. The user enters their credentials into the login interface. 2. The system verifies the submitted credentials. 3. The system grants access and displays the default dashboard view.
Alternative Flows	Invalid credentials → The system denies access and displays an authentication error message. Session timeout → The system automatically refreshes the session without interrupting the user.
Frequency of Use	High (Initiated upon every new user session or token expiry).
Priority	Critical (Gatekeeper for all subsequent RBAC routing).
Exception	Account locked due to repeated invalid attempts → Deny access, log event to security audit trail, and prompt system administrator intervention.
Postconditions	The user is successfully authenticated and an active session is established.

Table 3.2: Use Case - Manage Frameworks

Attribute	Description
Use Case ID	UC-02
Use Case Name	Manage Wazuh Telemetry Synchronization
Primary Actor	System Administrator
Preconditions	The Administrator is authenticated. <<include>> User Authentication. The external SIEM system is reachable.
Main Flow	1. The Administrator triggers a manual synchronization of security telemetry. 2. The system requests audit data from the external SIEM. 3. The system processes and maps the received data against the linked controls. 4. The system updates the compliance status for each control. 5. The system stores the audit results and notifies the Administrator.
Alternative Flows	SIEM system is unreachable → The system retries the connection and alerts the Administrator if it ultimately fails.
Frequency of Use	High (Continuous via Celery beat background tasks; occasional manual ad-hoc triggers by Superadmin).
Priority	Critical (Forms the core data ingestion pipeline for the compliance engine).
Exception	Wazuh REST API connection timeout or TLS handshake failure → Abort sync, log connection error, and queue retry via Celery exponential backoff.
Postconditions	The system reflects the latest compliance state based on the synchronized SIEM data.

Table 3.3: Use Case - Map Controls

Attribute	Description
Use Case ID	UC-03
Use Case Name	Map Framework Controls to Wazuh Rules
Primary Actor	System Administrator
Preconditions	The Administrator is authenticated. <<include>> User Authentication. Frameworks and controls exist in the system.
Main Flow	1. The Administrator selects a SIEM rule and links it to a specific compliance control. 2. The system validates the relationship between the rule and the control. 3. The system saves the mapping configuration. 4. The system confirms the successful linkage to the Administrator.
Alternative Flows	Invalid control selected → The system rejects the mapping and prompts the Administrator to select a valid control.
Frequency of Use	Low/Medium (Executed during initial platform setup or when regulatory frameworks release new versions).
Priority	High (Defines the relational logic for the entire compliance evaluation).
Exception	Mapping collision or invalid/deprecated Wazuh Rule ID → Prevent database commit, display strict validation error to the Superadmin.
Postconditions	A linkage is established between the SIEM rule and the compliance control for future audits.

Table 3.4: Use Case - Sync Telemetry

Attribute	Description
Use Case ID	UC-04
Use Case Name	Monitor Endpoint Compliance Status
Primary Actor	System Administrator (Trigger) / Celery (System Actor)
Preconditions	The user is authenticated. <<include>> User Authentication.
Main Flow	1. The user navigates to the dashboard view. 2. The system aggregates the latest compliance data. 3. The system visualizes the compliance score and detailed control statuses.
Alternative Flows	No data available → The system displays an empty state with a prompt to synchronize telemetry.
Frequency of Use	High (Serves as the primary operational dashboard for continuous monitoring).
Priority	High (Core visibility and monitoring function).
Exception	Endpoint agent disconnected or offline → Mark endpoint status as 'Stale/Offline', visually highlight the disconnection in the UI, and suspend compliance calculations for that node.
Postconditions	The user is presented with an up-to-date visual compliance posture.

Table 3.5: Use Case - View Dashboard

Attribute	Description
Use Case ID	UC-05
Use Case Name	Resolve Compliance Anomalies
Primary Actor	Security Auditor, System Administrator
Preconditions	The Auditor or Admin is authenticated. Compliance failures have been detected in the system.
Main Flow	1. The user identifies a non-compliant control on the dashboard. 2. The user initiates a resolution workflow and applies corrective actions to the endpoint. 3. The system verifies the change and logs the remediation status.
Alternative Flows	Resolution fails → The system maintains the 'Failed' state and prompts for manual investigation.
Frequency of Use	Medium (Event-driven, based on incoming alerts and failed states).
Priority	Medium/High (Critical for operational remediation, but dependent on external IT actions).
Exception	Insufficient RBAC permissions to apply resolution (e.g., Auditor attempting Admin action) → Deny action, return HTTP 403 Forbidden, and log unauthorized attempt.
Postconditions	The compliance anomaly is resolved and recorded in the historical audit ledger.

Table 3.6: Use Case - Export Audit Reports

Attribute	Description
Use Case ID	UC-06
Use Case Name	Export Audit Reports
Primary Actor	Security Auditor
Preconditions	The Auditor is authenticated. $\langle\langle\text{include}\rangle\rangle$ User Authentication. Compliance data exists.
Main Flow	1. The Auditor requests a formal compliance report for a specific framework. 2. The system compiles the historical audit data. 3. The system generates a formatted PDF document. 4. The system prompts the Auditor to download the document.
Alternative Flows	Generation failure $\rightarrow$ The system displays a generation error message and logs the failure.
Frequency of Use	Low (Triggered periodically based on audit cycles, e.g., monthly or quarterly).
Priority	High (Primary executive deliverable for external governance bodies).
Exception	PostgreSQL data retrieval timeout or PDF template rendering failure $\rightarrow$ Abort generation, display generic error modal, and log backend stack trace.
Postconditions	A timestamped audit report is delivered to the Auditor.

Chapter 4

Design

4.1 System Architecture

The Intelligent Compliance Management System (ICMS) uses an N-tier architecture. Separating the frontend, API, background workers, and database allows for scaling and debugging of individual components without affecting the entire system. Figure [4.1](#) shows the four main layers.

4.1.1 Client and Presentation Tier

The frontend is a React 18 Single Page Application (SPA). Moving the rendering logic to the browser reduces server load. This architecture also ensures that analysts can review compliance alerts without full page reloads.

4.1.2 Application and Middleware Tier

The backend uses the Django REST Framework. It functions as the primary gateway, verifying JWTs and enforcing role-based access before processing queries. Database traffic is routed through Django to maintain a consistent security boundary.

4.1.3 Asynchronous Processing Tier

Retrieving telemetry from the Wazuh SIEM is an I/O-intensive process. Executing these tasks on the main Django thread would cause API latency and timeouts. To prevent this, the system offloads I/O tasks to background Celery workers using Redis as the message broker.

4.1.4 Data Persistence Tier

PostgreSQL 18 is the primary database for its ACID guarantees. The schema uses relational tables for mapping frameworks to controls, while the JSONField type stores unstructured SIEM logs.

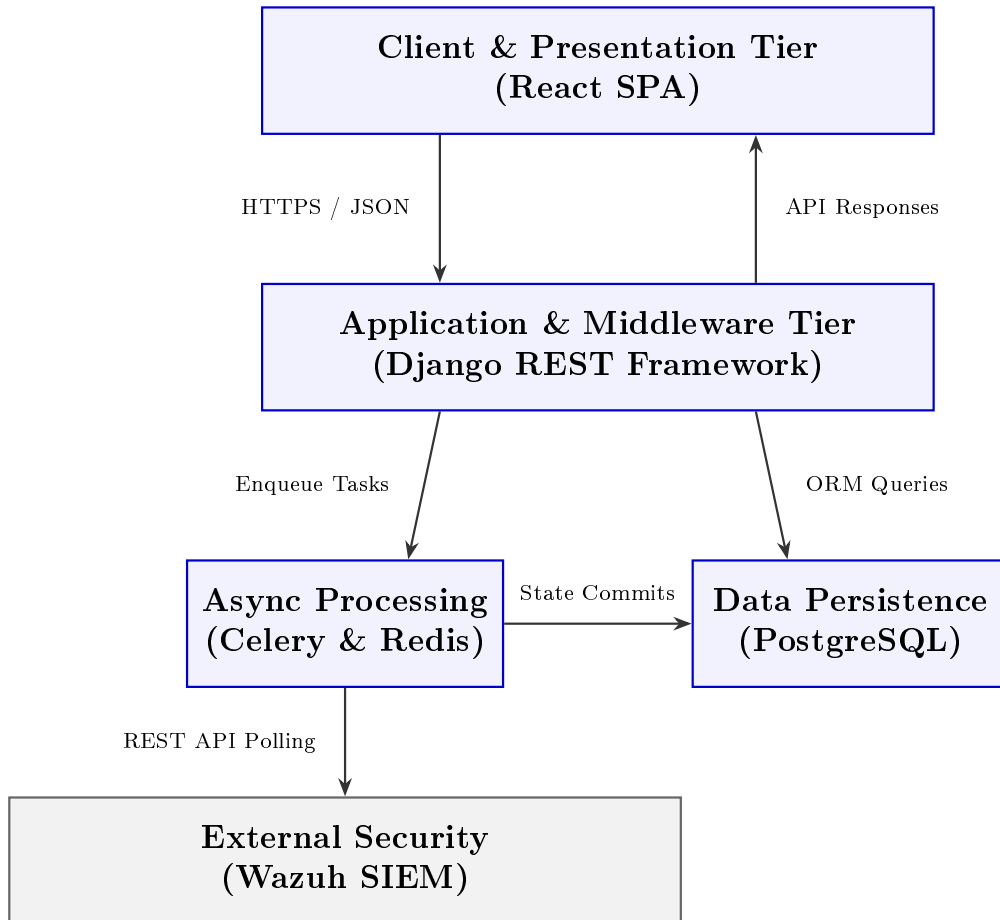


Figure 4.1: Tier-wise System Architecture of the ICMS Platform

4.2 Design Constraints

The design of the ICMS involved addressing several technical and environmental constraints.

4.2.1 Technical Constraints

For security, the database and Redis broker operate inside a private Docker bridge network. Their ports are not exposed to the host machine. Access to data requires requests

through the Django API container.

The integration with the Wazuh API introduced rate limits and network latency. Celery workers are configured to batch and throttle polling requests. Consequently, the dashboard displays the compliance state from the last successful synchronization rather than a real-time feed.

On the frontend, rendering large compliance grids can impact browser performance. Loading thousands of DOM nodes simultaneously can block the main thread. This is addressed by using React virtualization, which ensures the UI only renders visible elements.

4.2.2 Infrastructure and Operational Constraints

External factors, such as restricted access to university facilities, required the transition to developing and testing the architecture in a home-lab environment.

Running a full Wazuh SIEM manager locally is resource-intensive and can impact system stability. To manage this, a distributed configuration was used. One system was dedicated to running Wazuh, while another was used for development and as a monitored endpoint. Git was used to maintain consistency across different development environments while the primary infrastructure remained on a dedicated server.

4.3 Design Methodology

The platform was built using an iterative prototyping approach. The system was divided into distinct modules to allow for independent testing before integration.

The project was divided into four main phases:

- **Phase 1 (Prototyping):** The initial phase focused on verifying the core concept. This included setting up a basic Django backend and testing the Wazuh API to confirm successful retrieval of Security Configuration Assessment (SCA) telemetry.
- **Phase 2 (Core Architecture):** The second phase involved designing the PostgreSQL database schema to link frameworks to rules. JWT authentication and role-based routing were also implemented to secure the API.
- **Phase 3 (Asynchronous Telemetry):** Celery and Redis were added to handle heavy SIEM data ingestion. This established a background worker fleet for data

processing and deduplication.

- **Phase 4 (Frontend & Visualization):** The final phase involved building the React frontend. This included rendering data grids, implementing state management, and configuring the server-side PDF generator for audit reports.

4.4 High Level Design

The system architecture is represented through several views that cover software components, physical deployment, and operational sequences.

4.4.1 Component Diagram

Figure 4.2 shows the interactions between system modules. The React frontend sends HTTPS requests to the Django API Gateway. The IAM module verifies the JWT, and the Compliance Engine then translates the request into a database query. A separate Wazuh Polling Service retrieves alerts and stores them in PostgreSQL. The Wazuh Manager API also interacts with the Django API Gateway via Trigger Sync and provides HTTPS Telemetry to the Wazuh Polling Service.

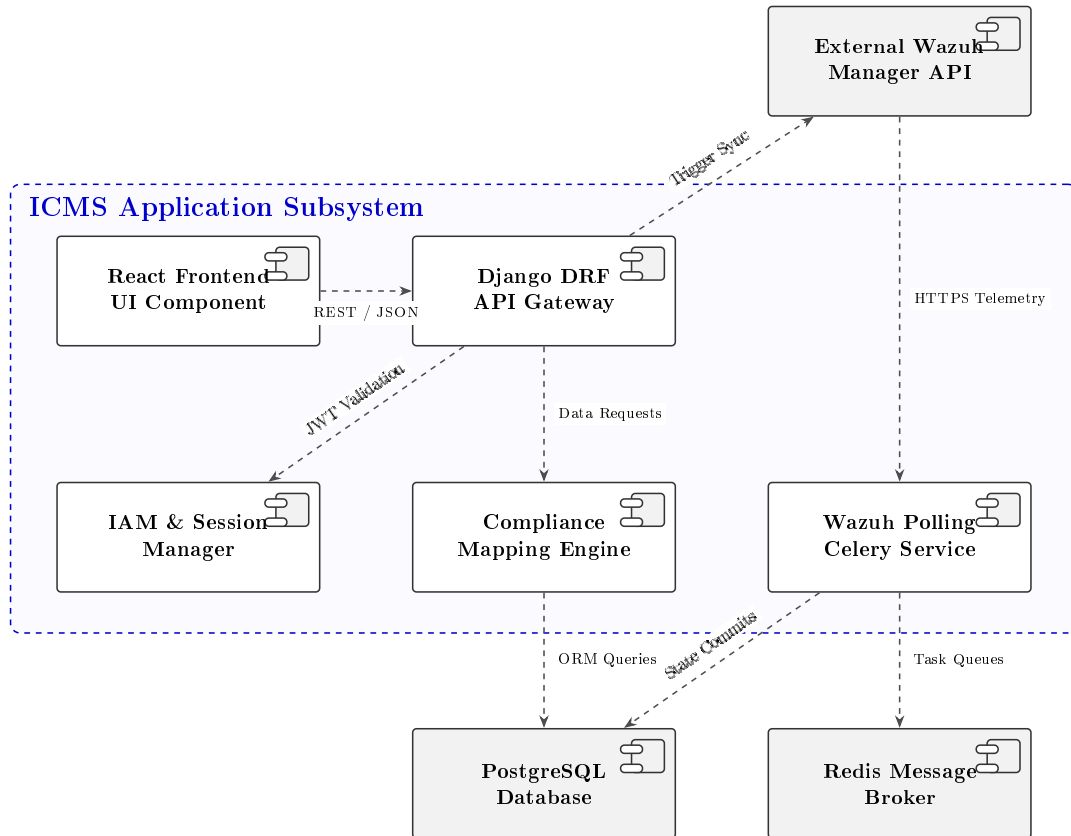


Figure 4.2: Conceptual View: UML Component Diagram

4.4.2 Deployment Diagram

Figure 4.3 outlines the Docker container environment. The Django backend, Celery workers, Redis, and PostgreSQL are deployed on the same host but isolated within a private bridge network. This configuration ensures the database remains inaccessible from external networks.

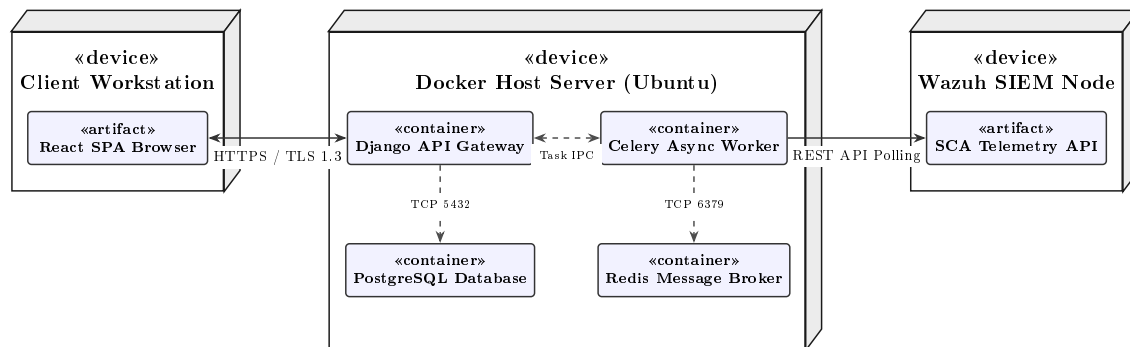


Figure 4.3: Physical View: UML Deployment Diagram

4.4.3 System Activity Diagrams

The activity diagrams below show the control flow for core platform features. Standard UML semantics, such as decision nodes and synchronization forks, are used to represent the system logic.

Authentication and IAM Routing

Figure 4.4 shows the login process. Django verifies credentials against PostgreSQL. If the verification fails, the system logs the event and denies the request. Upon successful authentication, the system issues a JWT and evaluates the user's role. Administrators are directed to configuration settings, while Auditors are restricted to read-only dashboards.

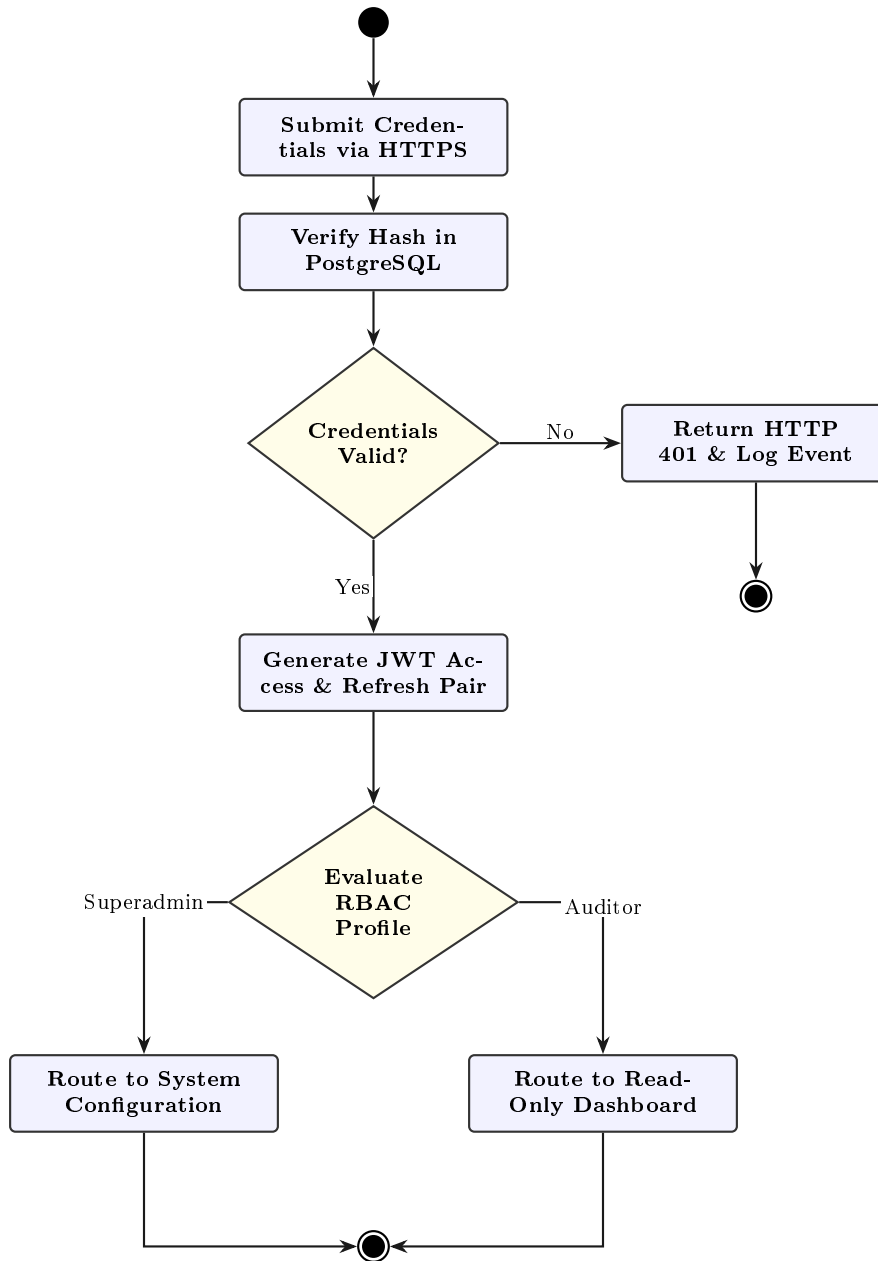


Figure 4.4: Activity Diagram: Authentication and IAM Routing

Asynchronous Telemetry Ingestion

Figure 4.5 shows the asynchronous ingestion process. When a synchronization task is triggered, the main thread returns an HTTP 202 status to the client. Simultaneously, the Celery worker polls the Wazuh API. If the SIEM is unreachable, the system uses exponential backoff. Once telemetry is retrieved, the system parses the JSON data and updates the database.

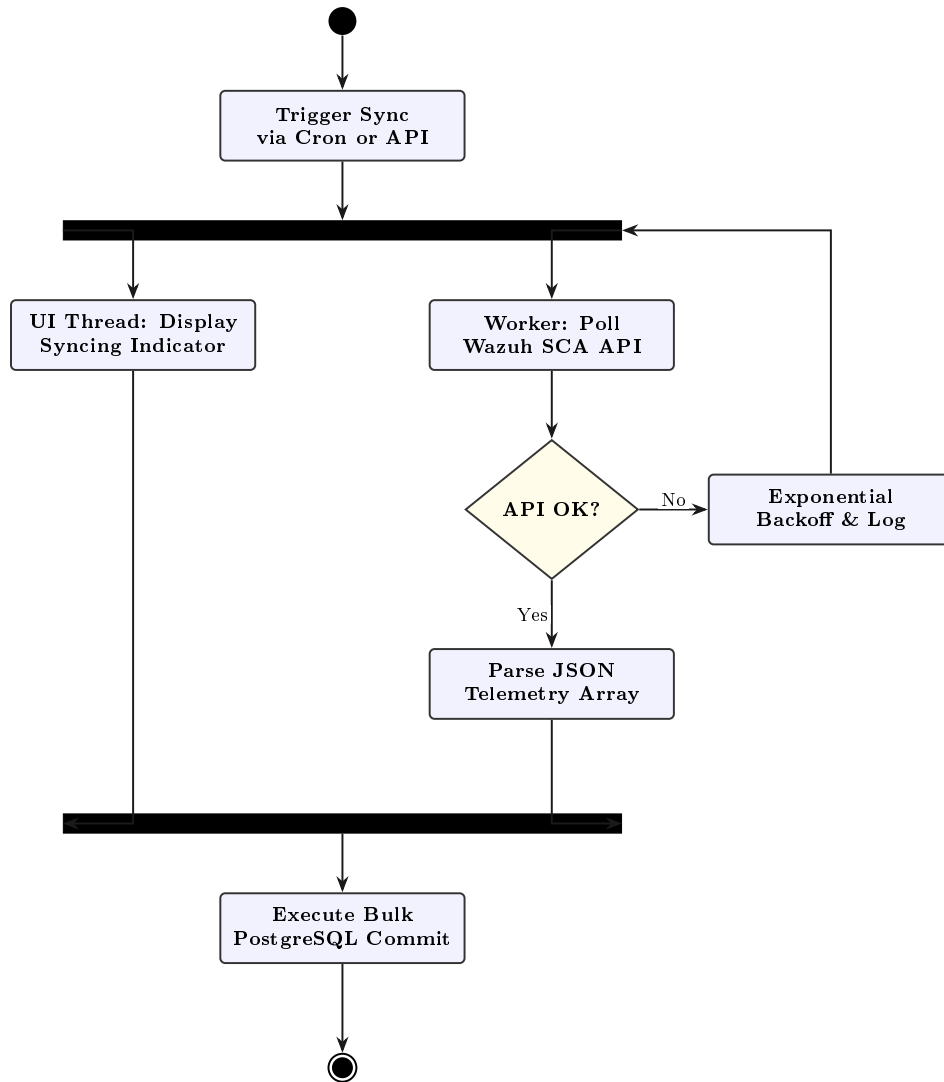


Figure 4.5: Activity Diagram: Parallel Asynchronous Telemetry Ingestion

Compliance State Evaluation

The mapping of technical alerts to policies like ISO 27001 is shown in Figure 4.6. The system uses a worst-case priority algorithm; if a technical check fails on a single node, the corresponding governance control is marked as non-compliant. This evaluation continues until all policies are processed.

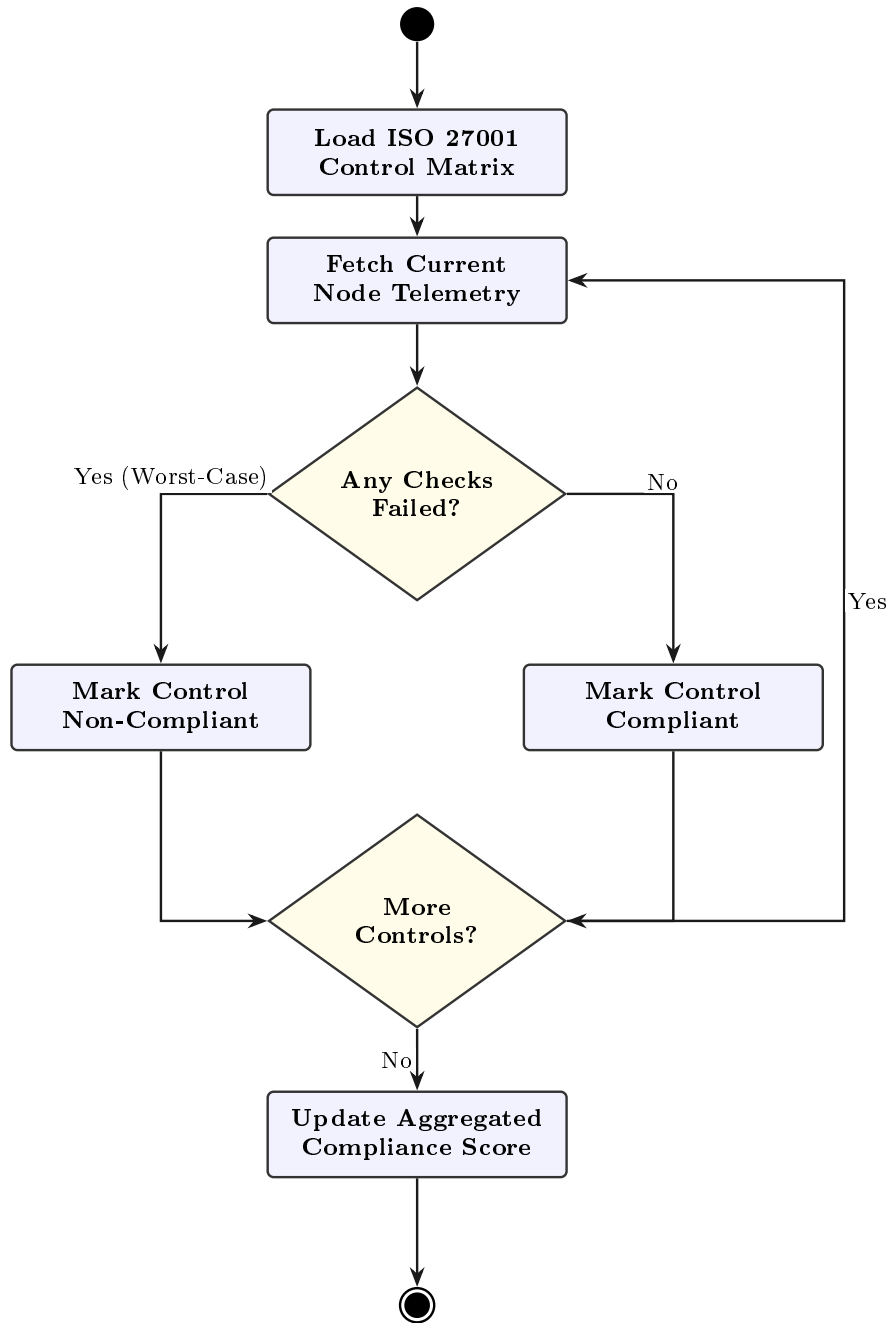


Figure 4.6: Activity Diagram: Iterative Compliance Evaluation and Worst-Case Priority

Audit Report Generation

The audit report generation process is automated, as shown in Figure 4.7. When a user selects a framework, the backend aggregates relevant database records. This data is injected into an HTML template, and a PDF is generated on the server for delivery to the client.

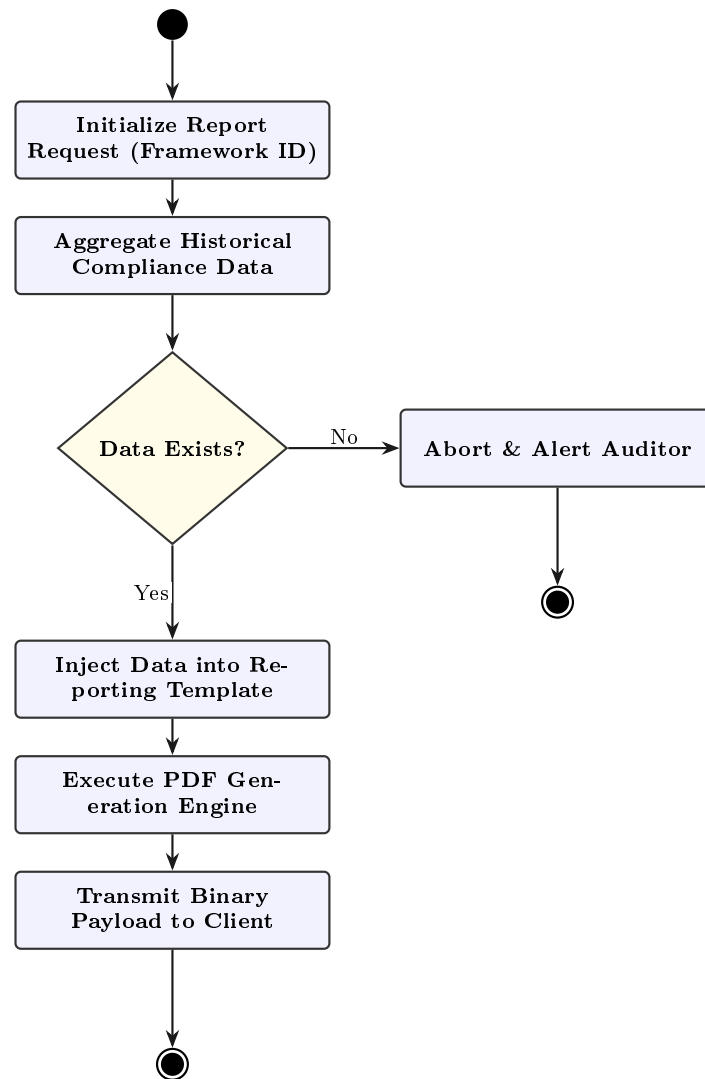


Figure 4.7: Activity Diagram: Automated PDF Audit Report Generation

Framework Rule Mapping and Validation

Administrators map Wazuh rules to framework controls as shown in Figure 4.8. Before saving the mapping, the system queries the SIEM API to verify the rule ID. If valid, the system checks for existing mappings in the database. This validation prevents accidental overwriting of existing configurations.

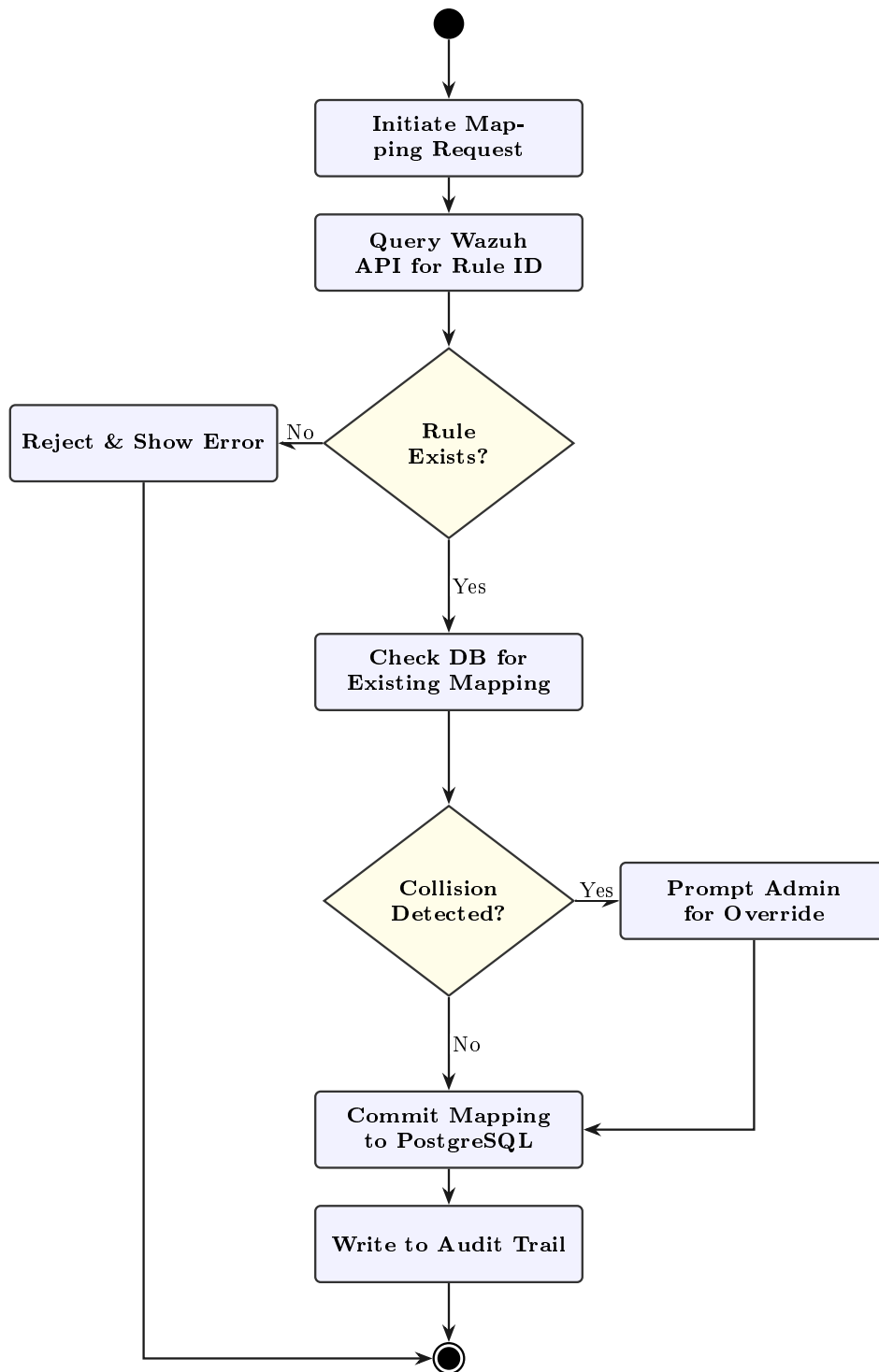


Figure 4.8: Activity Diagram: Framework Rule Mapping and API Validation

Real-Time Alerting and Notification

The platform includes an alerting pipeline as shown in Figure 4.9. When a compliance check fails, the engine evaluates its severity. Minor issues are recorded in the database logs, while critical failures trigger automated notifications via email and webhooks to

auditors.

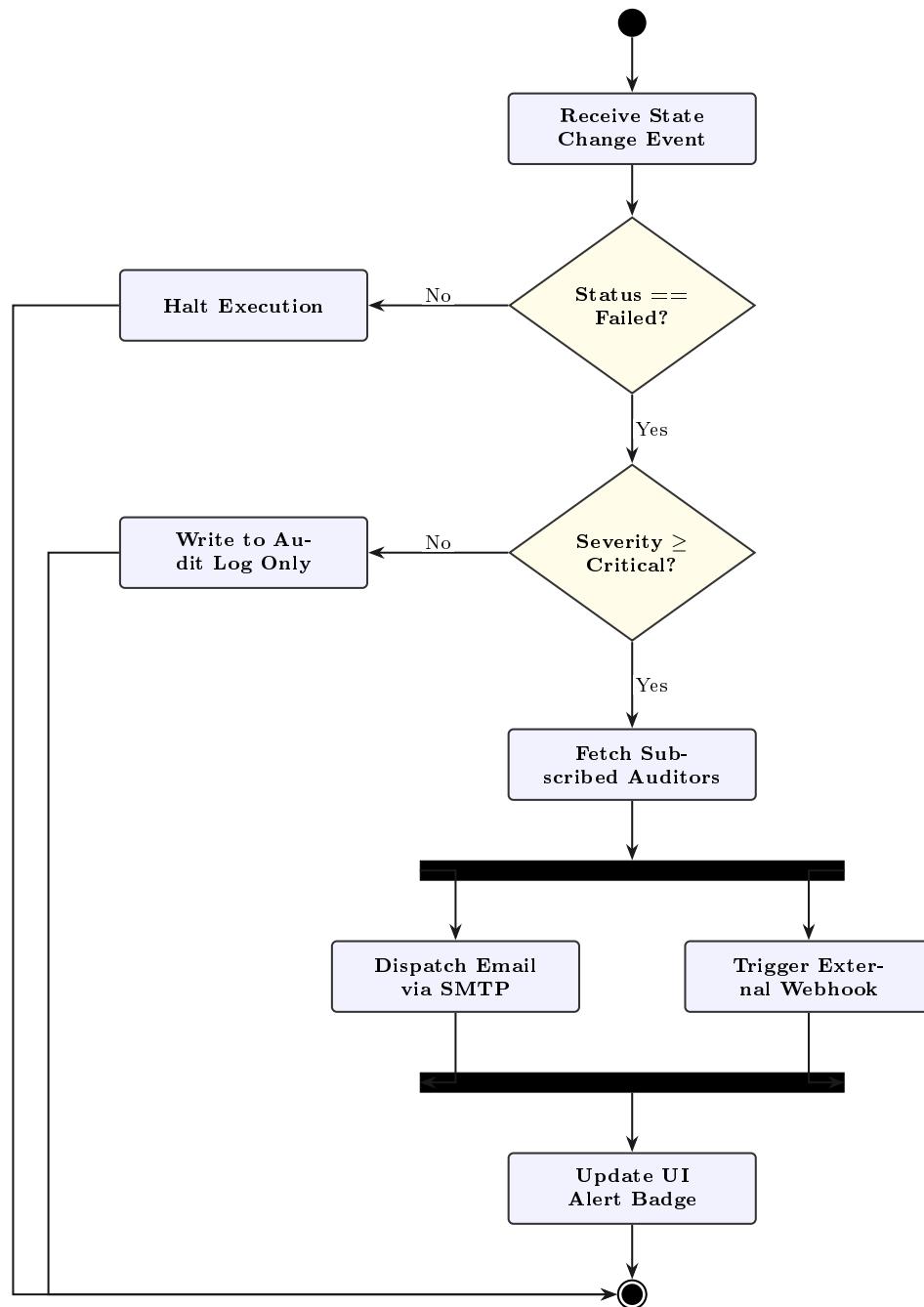


Figure 4.9: Activity Diagram: Real-Time Alerting and Notification Fan-out

4.4.4 System Sequence Diagrams

The sequence diagrams below show the interactions between internal system components over time.

Authentication and JWT Issuance

Figure 4.10 shows the API calls during the login process. If the credentials match, the IAM module issues a JWT. If the verification fails, the system returns an HTTP 401 Unauthorized response.

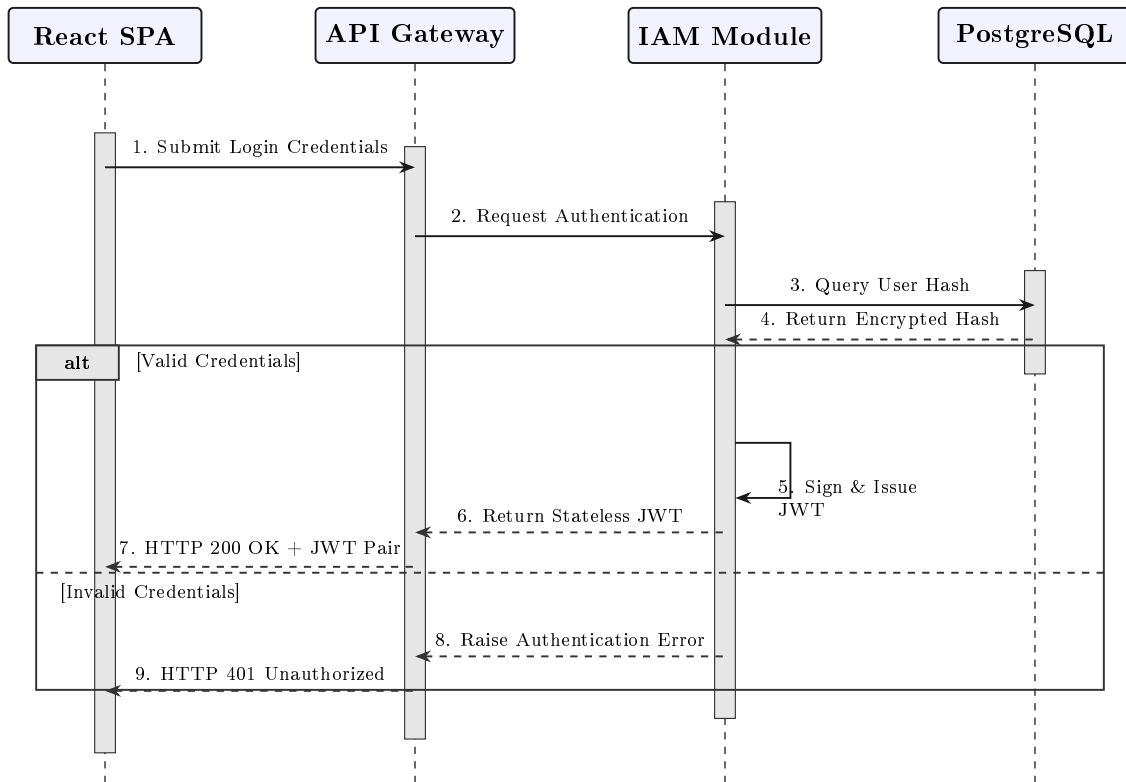


Figure 4.10: Sequence Diagram: Authentication Flow with UML Alt Fragment

Asynchronous Telemetry Ingestion

The telemetry synchronization uses a bulk update pattern to optimize database performance, as shown in Figure 4.11. The Celery worker retrieves endpoint states from Wazuh, processes the impacts on the regulatory framework, and commits the results to PostgreSQL in a single transaction.

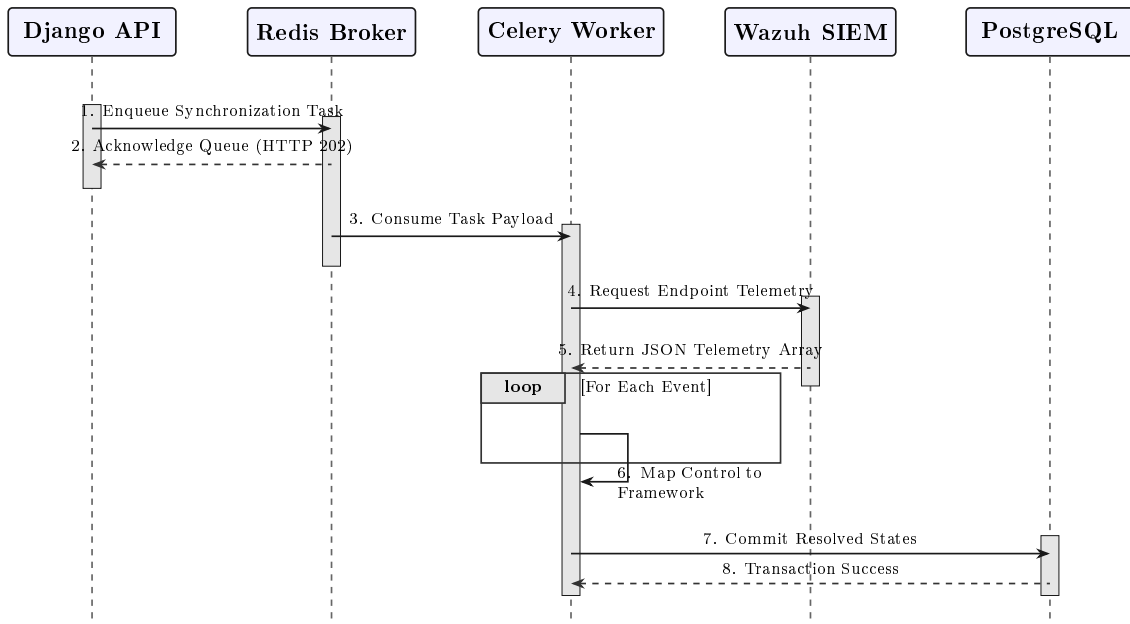


Figure 4.11: Sequence Diagram: Background Telemetry Synchronization with UML Loop

Framework Configuration and Rule Mapping

Figure 4.12 shows the API flow for linking Wazuh rules to policies. The backend verifies permissions and records the mapping in the database. This synchronous process ensures that subsequent telemetry processing can utilize the new rule associations immediately.

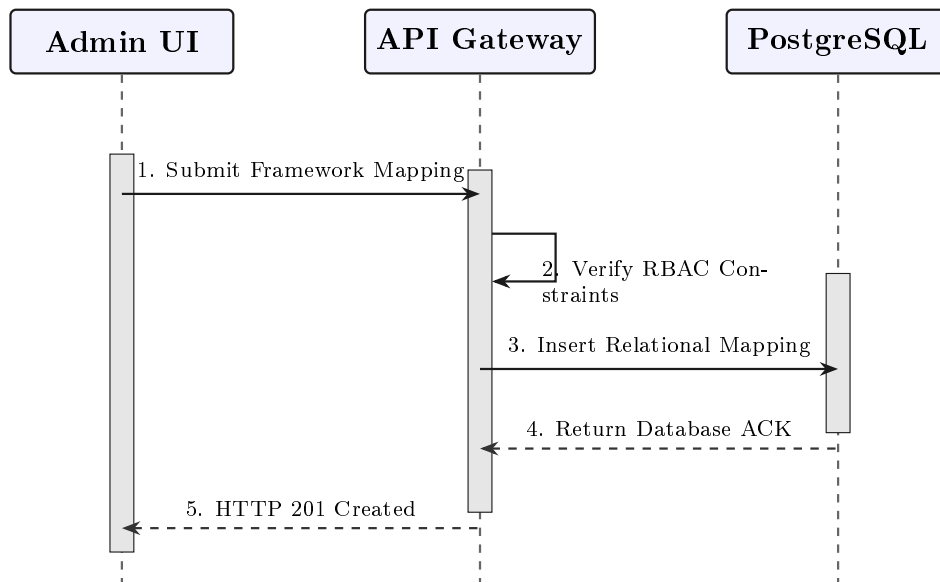


Figure 4.12: Sequence Diagram: Framework Rule Mapping

Audit Report Generation

Figure 4.13 outlines the PDF generation process. If the database query returns no records, the system terminates the request. If data is present, the reporting engine generates the PDF binary and streams it to the client.

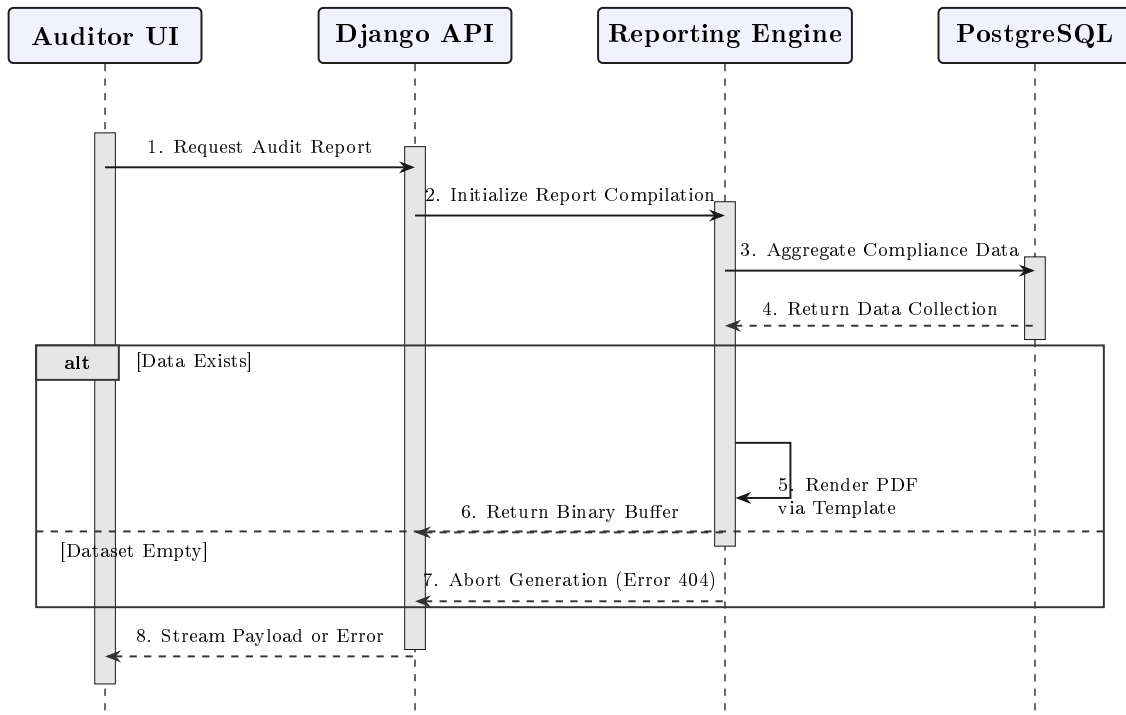


Figure 4.13: Sequence Diagram: Report Generation with Alternative Data States

4.5 Low Level Design

The Low-Level Design (LLD) translates the high-level system architecture into fundamental software building blocks. This section maps the specific object-oriented structures, persistence models, and relational bindings that dictate the execution of the ICMS platform's core business logic.

4.5.1 UML Class Diagram

Figure 4.14 shows the Django ORM classes and their relationships. Relational constraints link regulatory frameworks to specific endpoint telemetry logs.

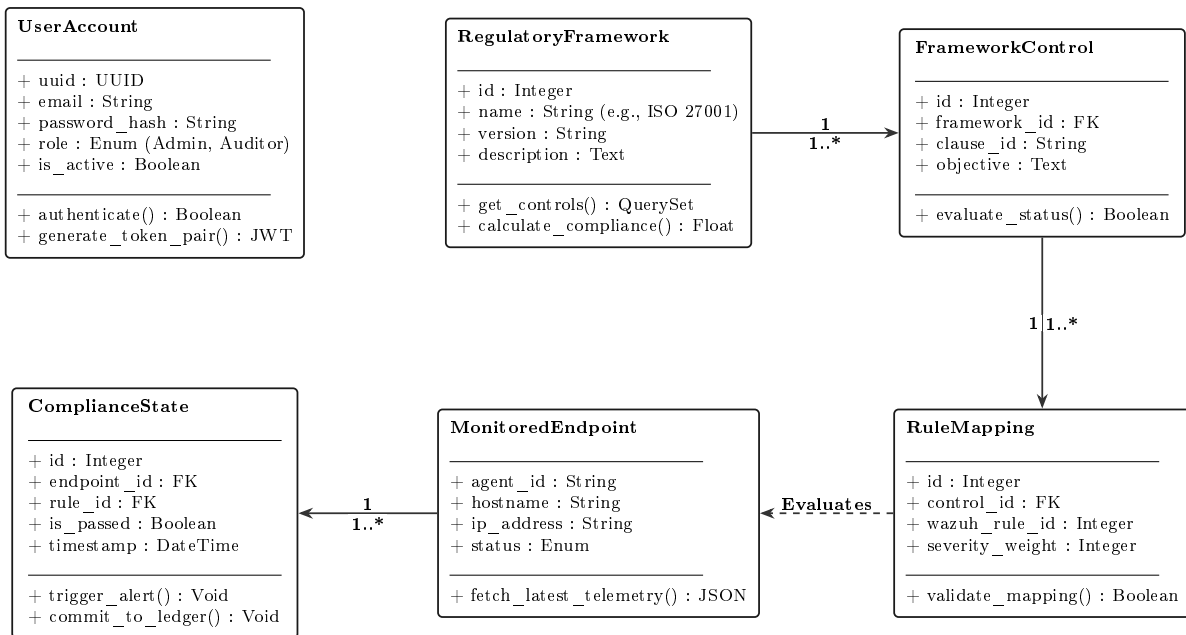


Figure 4.14: UML Class Diagram: Core Object-Relational Models

4.6 Database Design

I built the database on PostgreSQL. The tables are normalized to the Third Normal Form (3NF) to prevent update anomalies. When you're pulling in thousands of SIEM alerts an hour, you need strict ACID compliance so the data doesn't corrupt.

4.6.1 Physical Data Model (Core ERD)

Figure 4.15 shows the actual physical schema. I mapped out the exact column types and foreign keys we used to build the PostgreSQL database.

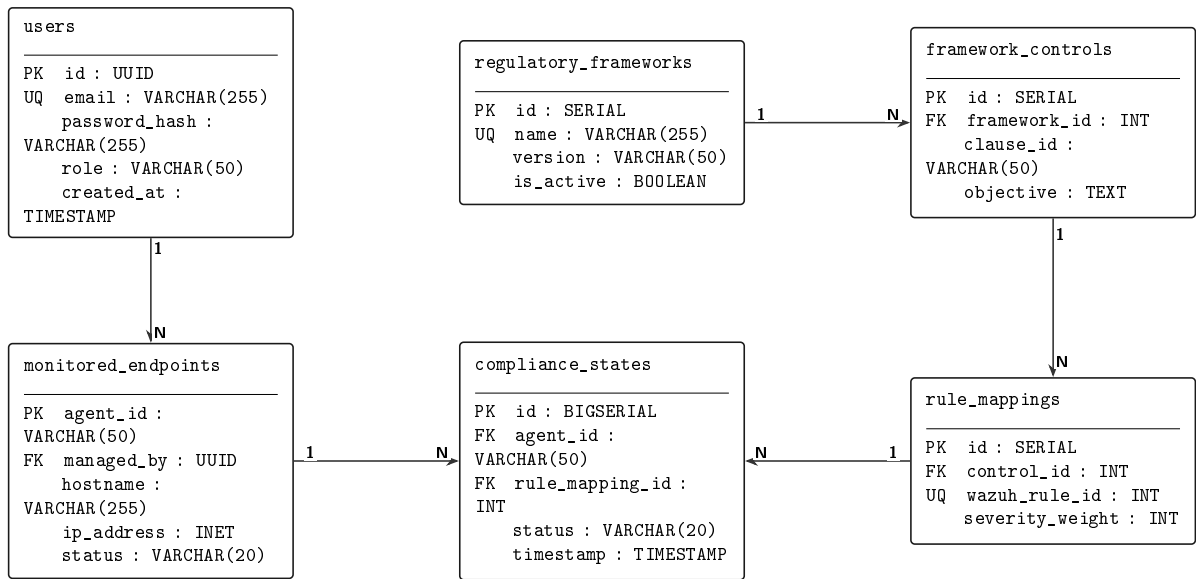


Figure 4.15: Physical Database Schema: Core Relational Data Model with Exact Data Types

4.6.2 Extended Entity Relationship Diagram (EERD)

Figure 4.16 is an Extended ERD (EERD) that visualizes the logical rules we put in place.

Notice the disjoint (**d**) specialization: a generic User Account splits strictly into either a System Admin or a Security Auditor. That handles our RBAC at the database level. I also modeled the Compliance State as a weak entity, because an alert record is useless without its parent Monitored Endpoint.

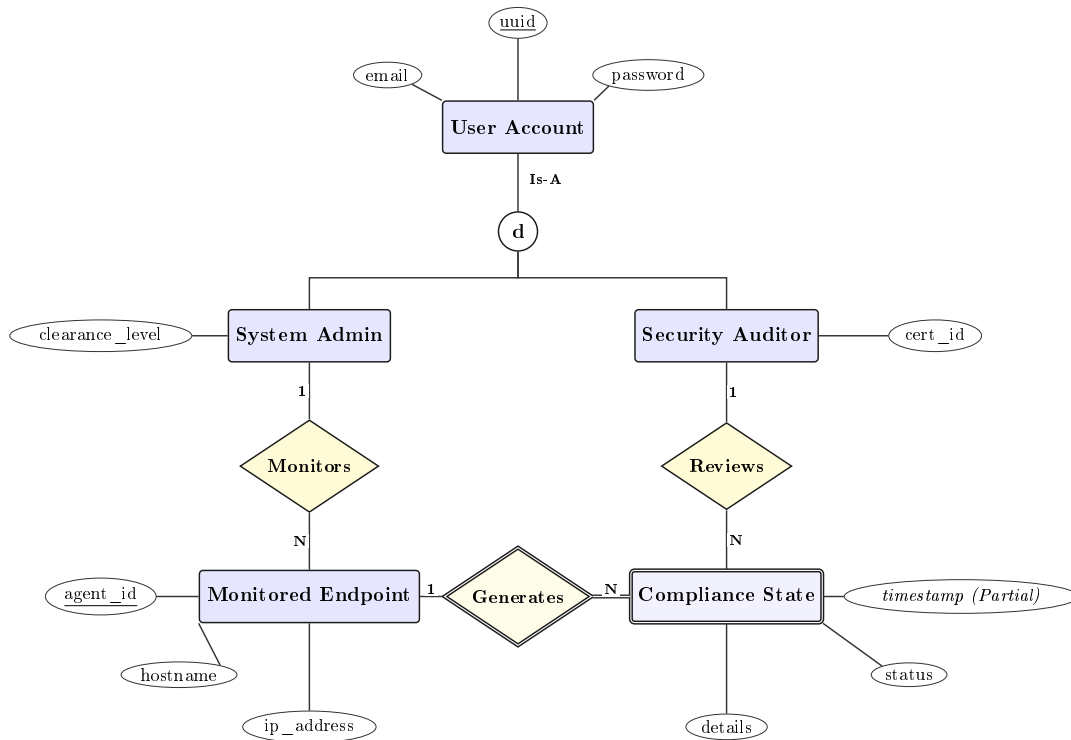


Figure 4.16: Extended Entity Relationship Diagram (EERD) with Constraints and Attributes

4.7 Graphical User Interface (GUI) Design

For the frontend, I built a React Single Page Application (SPA). I wanted the interface to be clean and data-focused so analysts don't get overwhelmed by noise. The screenshots below show the default dark mode, though the app supports theme switching.

4.7.1 Authentication and Identity Management

The login screen is bare-bones by design. The user submits their credentials, the backend validates the JWT, and they get routed to their specific dashboard based on their role.

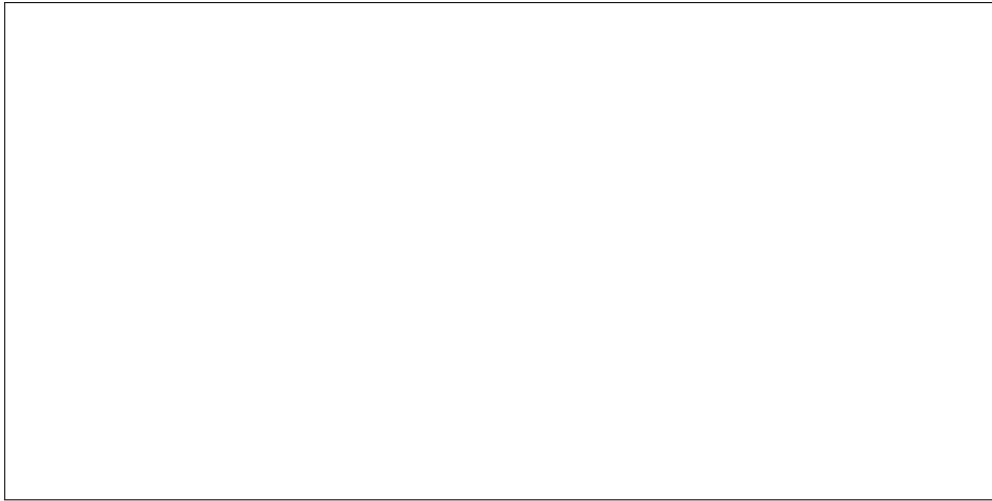


Figure 4.17: GUI Design: Secure User Authentication Portal

4.7.2 Superadmin Configuration Portal

Superadmins have full read/write access. Their dashboard gives them complete control over the Wazuh pipeline, the framework rules, and user accounts.

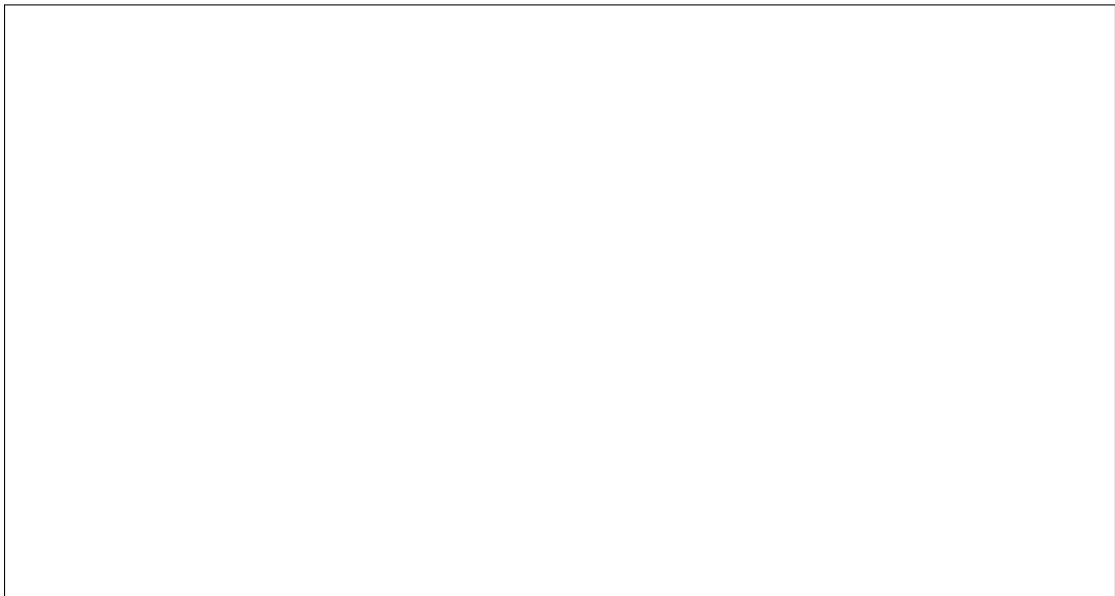


Figure 4.18: GUI Design: Superadmin Global Compliance Dashboard

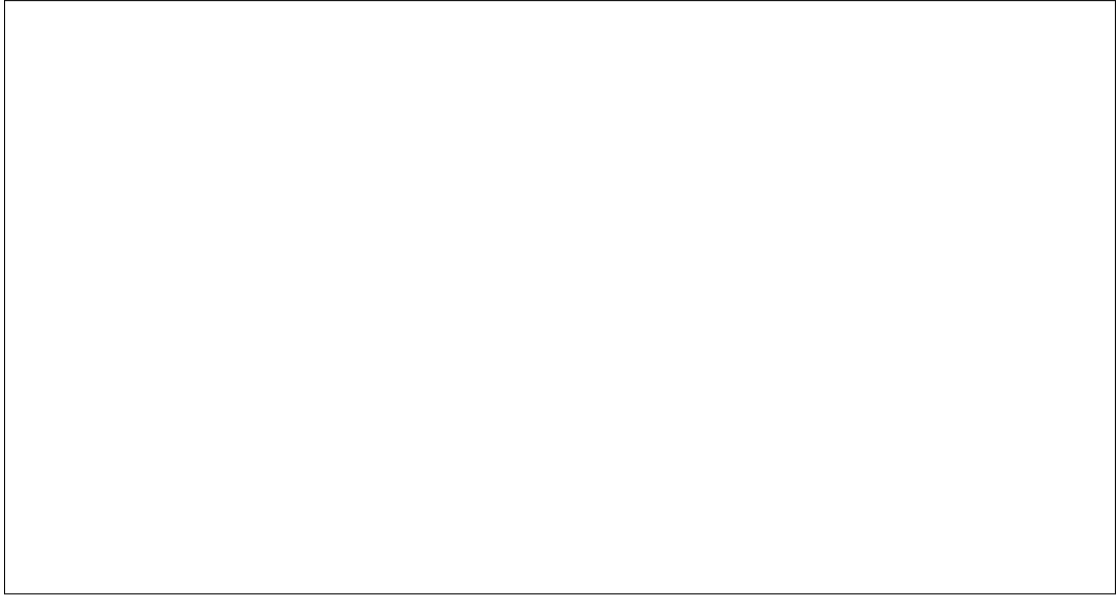


Figure 4.19: GUI Design: Framework and Wazuh Rule Mapping Interface

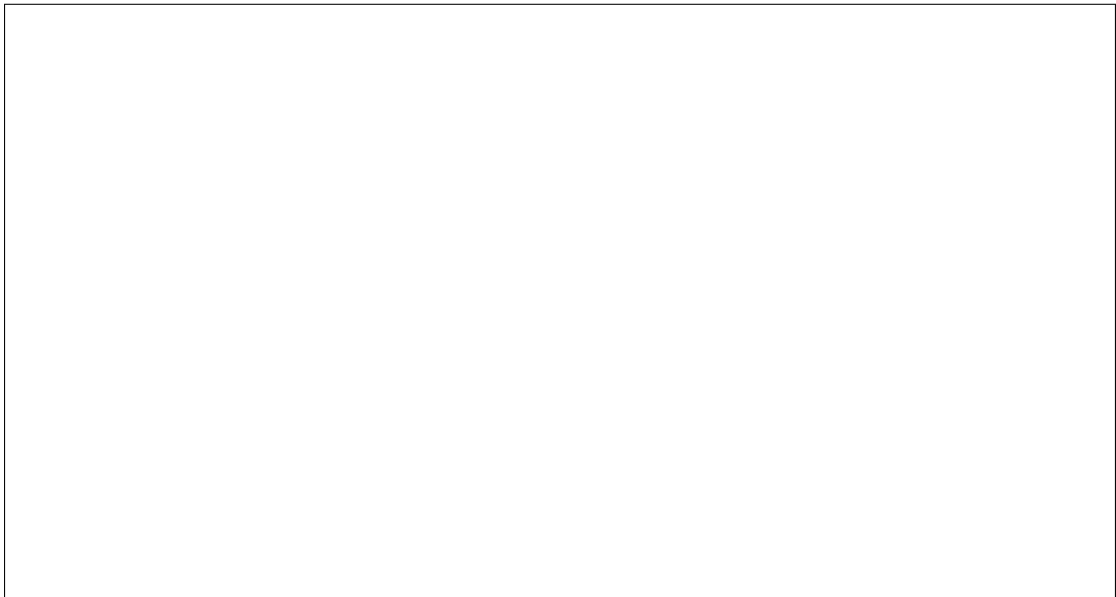


Figure 4.20: GUI Design: Monitored Endpoints and Telemetry Synchronization Status

4.7.3 Admin and Auditor Workspaces

The React app changes what it renders depending on who logs in. System Admins can manage endpoints but can't touch the compliance frameworks. Security Auditors get a strict read-only view, ensuring they can't accidentally alter the audit evidence.

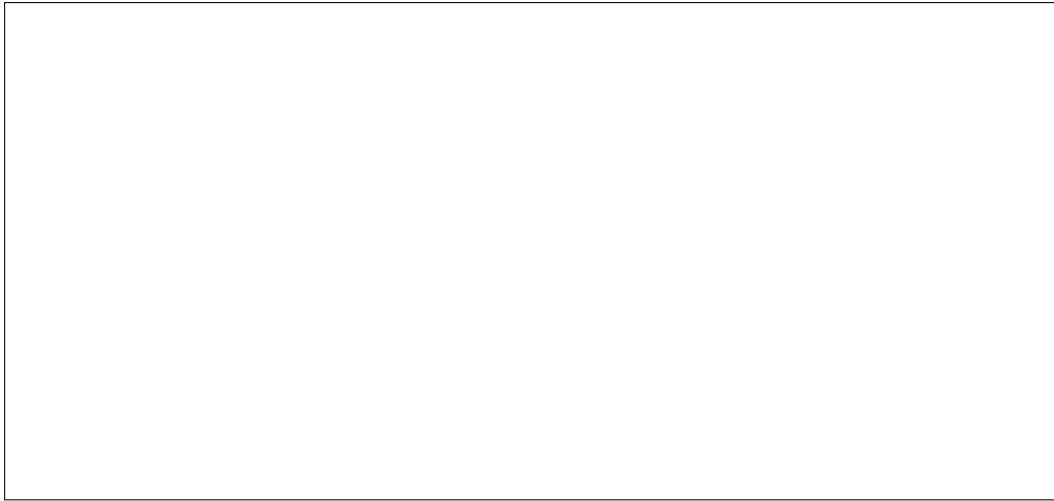


Figure 4.21: GUI Design: System Admin Operational Dashboard

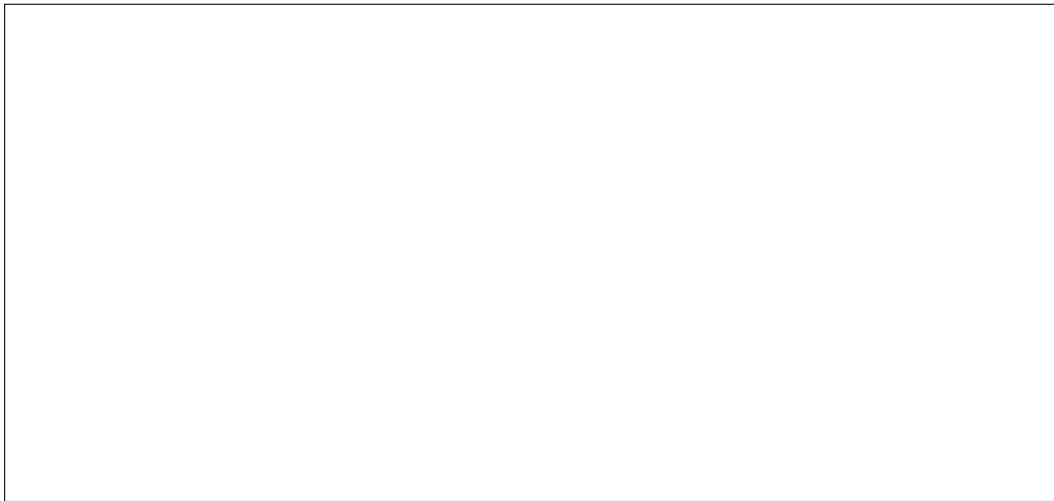


Figure 4.22: GUI Design: Security Auditor Read-Only Compliance View

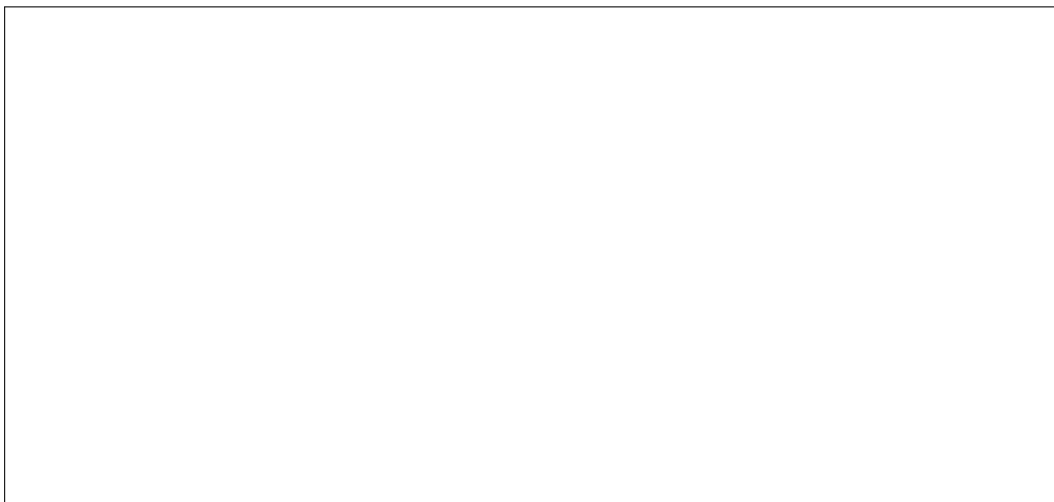


Figure 4.23: GUI Design: Auditor Report Generation Modal

4.7.4 Automated PDF Audit Artifacts

External auditors need physical files, so I built a PDF rendering engine on the backend. It aggregates the PostgreSQL data and generates formal reports like the ones below.

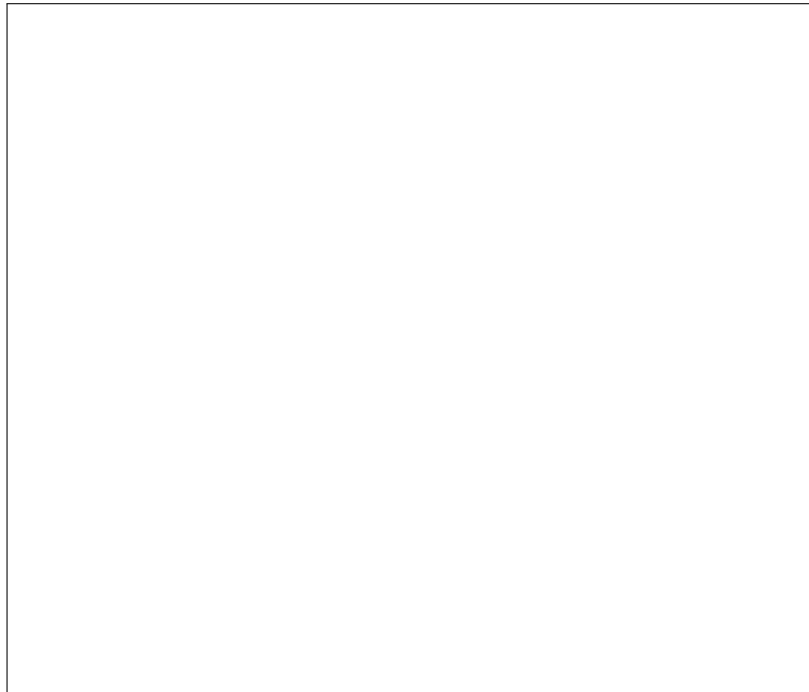


Figure 4.24: System Output: Automated PDF Report - Cover and Executive Summary

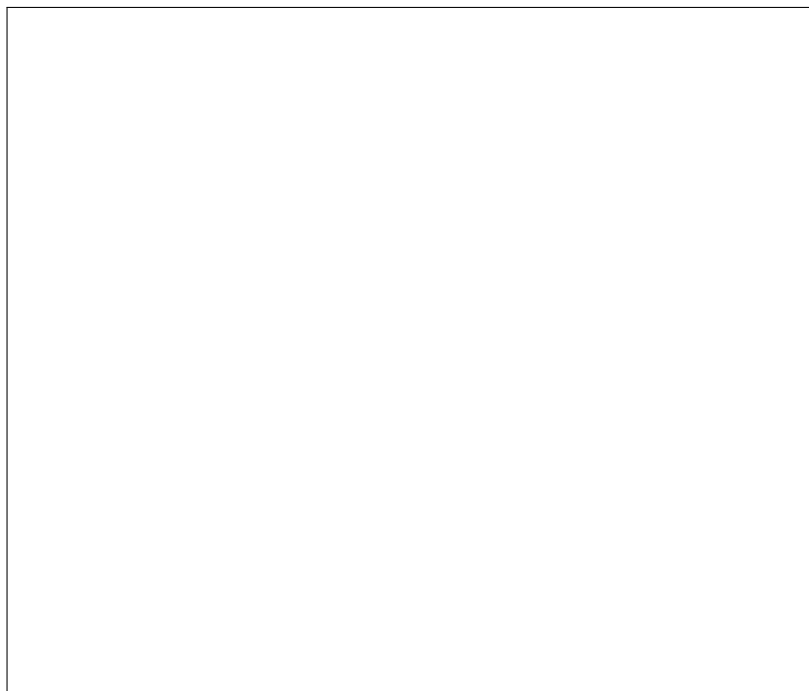


Figure 4.25: System Output: Automated PDF Report - Aggregated Framework Control Matrix

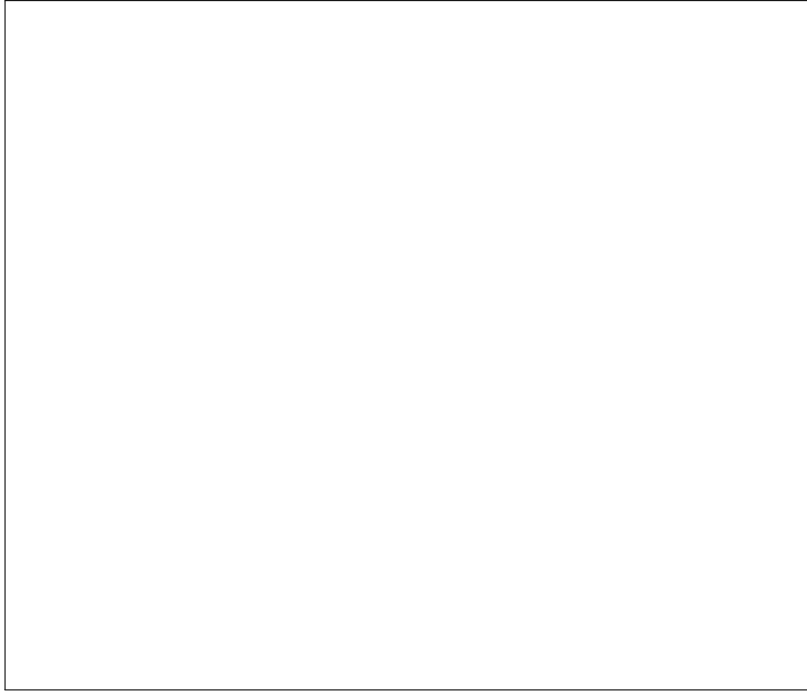


Figure 4.26: System Output: Automated PDF Report - Technical Wazuh Telemetry Evidence

4.8 External Interfaces

The ICMS doesn't run in a vacuum. It talks to several external services. This section lists out the protocols and authentication methods I used to keep those connections secure.

4.8.1 Wazuh SIEM REST API

The whole system relies on raw logs coming from the Wazuh Manager. My Celery workers hit the Wazuh REST API over HTTPS. I secured this with Bearer tokens generated by the Wazuh server. We pull down the JSON payloads and validate them against the Django ORM before saving them.

4.8.2 SMTP Notification Gateway

For real-time alerts, the backend constructs HTML emails and fires them off through an external SMTP server over port 587 using STARTTLS. I threw this logic into the async message broker so it wouldn't block the API Gateway.

4.8.3 Outbound Webhook Integration

I also added outbound webhooks so the system can talk to tools like Slack or Microsoft Teams. Whenever a compliance rule breaks, it shoots out an HTTP POST request with the JSON event data. To prevent spoofing, I added an HMAC-SHA256 signature to the headers.

4.8.4 Interface Specification Matrix

Table 4.1 summarizes the technical parameters and boundary constraints of the platform's external interfaces.

Table 4.1: External Interface Technical Specifications

Interface Target	Network Protocol	Data Format	Authentication Mechanism	Directionality
Wazuh Manager API	HTTPS (TCP 443)	JSON	Bearer Token / API Key	Bidirectional (Polling)
SMTP Relay Server	SMTP (TCP 587)	MIME / HTML	SMTP AUTH (TLS)	Outbound Only
Enterprise Webhooks	HTTPS (TCP 443)	JSON	HMAC-SHA256 Signatures	Outbound Only
Client Web Browser	HTTPS (TCP 443)	HTML / JSON	JWT (Stateless)	Bidirectional

Chapter 5

Implementation

The design phase established the requirements for the ICMS platform and the relationships between its components. This chapter documents the implementation process. Translating the architectural diagrams from Chapter 4 into code involved addressing hardware constraints, Docker networking, and the integration of a Python backend, a JavaScript frontend, and an external SIEM on a separate machine. The following sections describe each layer of the implementation.

Development used consumer-grade hardware in a distributed home-lab setup. The Wazuh SIEM manager operated on a dedicated laptop at a fixed IP address ([192.168.100.100](#)). The backend, frontend, and database stack was containerized on a second machine using Docker Compose. Every service configuration, environment variable, and migration step described here corresponds to the project repository.

5.1 System Architecture

As established in the design phase (as shown below in Figure [5.1](#)), the ICMS platform follows an N-tier client-server architecture.

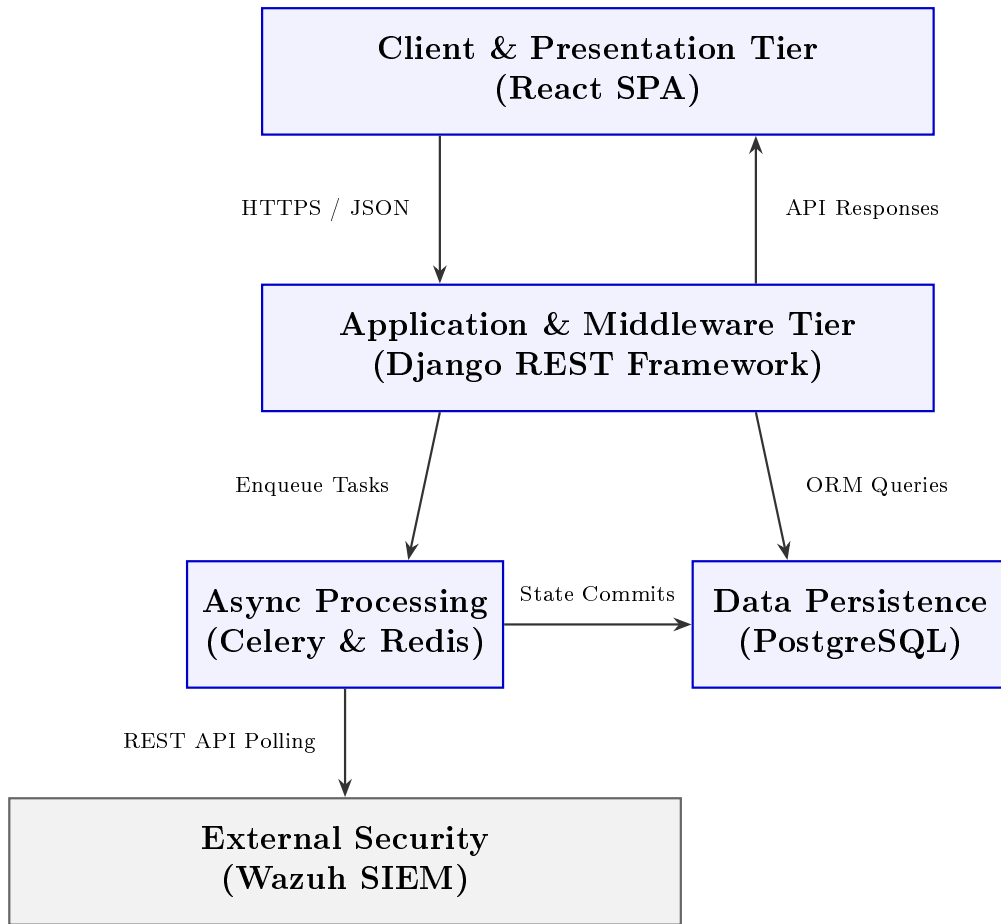


Figure 5.1: Implemented N-Tier System Architecture

In practice, this mapped onto two concrete code repositories: a `backend/` directory containing the Django application and all its sub-apps, and a `frontend/` directory containing the React SPA. Both are wired together through a `docker-compose.yml` file at the project root that defines seven services and a shared bridge network.

The full service topology (as shown below in Figure 5.2) illustrates the container environment and network isolation.

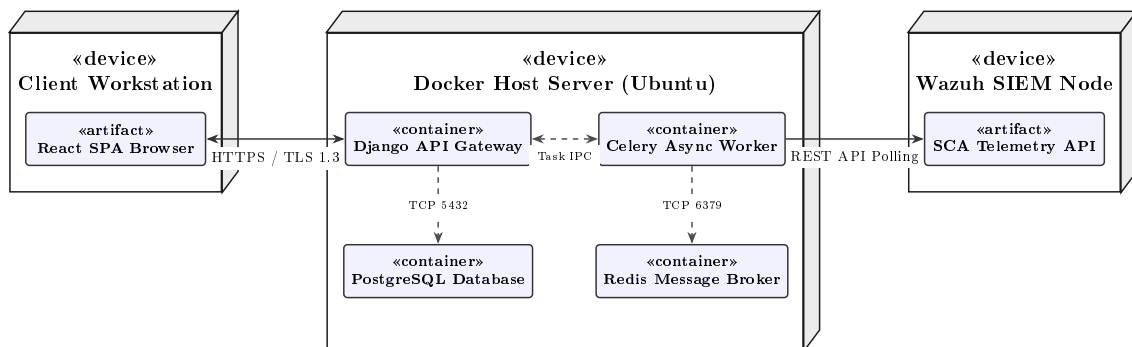


Figure 5.2: Implemented Service Topology and Container Networking

When the stack boots, Docker brings up the database and Redis first, waits for their health checks to pass, and only then starts the Django backend. This startup ordering was not optional; without it, the backend would crash on boot trying to run migrations against a database that was not yet accepting connections. The Celery worker and Celery Beat scheduler also depend on the backend image and start only after Redis is confirmed healthy. Wazuh, operating as the SIEM data source, is deployed independently; prior work on ML-augmented Wazuh deployments confirms its suitability as a telemetry engine for configuration assessment workloads [15].

5.1.1 Presentation Layer Implementation (Frontend)

The frontend is a React 19 Single Page Application built with Vite 8. Vite was chosen over older toolchains because it uses native ES modules during development, which reduces cold-start times to under 200ms. Routing is handled by React Router v7, which uses a declarative `<Route>` structure to enforce authentication at the component level.

Every HTTP request to the Django backend uses an Axios instance configured with interceptors. The request interceptor reads the JWT access token from `localStorage` and appends it as a `Bearer` header. The response interceptor monitors for `401 Unauthorized` responses; when one occurs, it triggers a token refresh. If the refresh fails, the system clears the session and redirects to the login page. This centralization removes the need for token-refresh logic in individual components.

The UI is styled with Tailwind CSS v4 using a custom `@theme` token set. Recharts is used for data visualization, including the compliance score gauge and per-framework bar charts. Icon assets are provided by Lucide React. The routing table also enforces RBAC at the frontend level: the `/manage-admins` route requires the `super_admin` role, ensuring the UI access matches the API permissions.

5.1.2 Middleware and API Gateway (Backend)

The backend is a Django 5.1 project with an installed application, `compliance`, that contains the models, views, serializers, and management commands. Project configuration is in `core/settings.py`, and secrets are loaded via `python-decouple` from a `.env` file.

Authentication uses `django-restframework-simplejwt`. Access tokens are valid for 24 hours, and refresh tokens last seven days with rotation enabled. The `CustomTokenObtainPairView` adds a two-factor authentication gate: if the user has TOTP enabled via `pyotp`,

the view returns a `requires_2fa` flag, requiring a second verification step before tokens are issued.

The `compliance` app's `views.py` includes DRF ViewSets for standard resources and custom `APIView` subclasses for dashboard aggregation and PDF report generation. The `DashboardSummaryView` uses an annotated queryset with `DISTINCT ON agent_id` to find the latest scan per agent, then computes compliance counts using Django ORM `Count()` with `Q()` filter objects. This optimization processes the results in a single SQL statement.

5.1.3 Asynchronous Processing Engine

Compliance scans involve HTTP polling against the external Wazuh REST API. Running these tasks on the main Django request thread would block the server. Instead, work is dispatched to a Celery worker process in a separate container.

The Celery application is initialized in `core/celery.py`. It reads configuration from Django settings and uses `autodiscover_tasks()` to locate `tasks.py` files. The system uses the `run_scheduled_wazuh_sync` task in `compliance/tasks.py`, which includes a retry policy for API failures. A network error triggers an automatic 60-second backoff and retry.

Redis operates in a separate container and is the message broker and result backend. The Celery Beat scheduler uses the `DatabaseScheduler` from `django-celery-beat`. Scan frequency is stored in the PostgreSQL database and can be modified through the UI without restarting containers. When a manual scan is triggered, the `/api/run-scan/` endpoint returns a 202 `Accepted` response and runs the task in the worker container.

5.1.4 Data Persistence Layer

PostgreSQL 18 is the primary database, running in its own container and accessible to the backend over the Docker bridge network. This network isolation approach, where the persistence tier is bound to a private bridge and never exposed to a host port, is a recognized security pattern for containerized applications [12]. Django connects via `psycopg[binary]` version 3.2. Built-in persistent connection support handles connection reuse.

The schema is managed through Django ORM migrations. The backend container entrypoint script runs `python manage.py migrate` before starting the server to ensure the

schema is current. Initial seed data, including regulatory controls and Wazuh rule mappings, is loaded via a management command that creates the relational records from a structured data file.

Raw Wazuh SCA check data is stored in a `JSONField` on the `ScanResult` model. Storing the variable-structure SCA response as JSON preserves the original audit evidence while extracted compliance metrics are stored in relational columns for dashboard rendering. Research into JSON versus column-oriented storage confirms that this hybrid approach provides field-level flexibility without sacrificing the indexing and aggregation benefits of the relational engine [8].

5.1.5 Architectural Technology Stack

Table 5.1 summarises the full technology stack across all four implementation tiers, mapping each component to its specific library or service and its role in the system.

Table 5.1: Architectural Technology Stack and Implementation Roles

Architecture Tier	Technology	Version	Implementation Role
Client (Frontend)	React	19.0.0	Single Page Application (SPA) UI layer. Manages client-side routing and state without full page reloads.
Client (Frontend)	Vite	8.0	Frontend build tool and development server, replacing Webpack for faster Hot Module Replacement (HMR).
Client (Frontend)	Recharts	3.8	Data visualization library used to render the compliance metrics and endpoint status dashboards.
Middleware (Backend)	Django	5.1	Core Python framework managing the ORM, database migrations, and overarching project structure.
Middleware (Backend)	Django REST Framework	3.15	Serializes Django models into JSON payloads and handles ViewSet routing for the React frontend.
Middleware (Backend)	SimpleJWT	5.3	Handles stateless authentication, issuing 24-hour access tokens and 7-day rotating refresh tokens.
Middleware (Backend)	xhtml2pdf	0.2.16	Generates the physical PDF audit reports directly from Django HTML templates on the server side.
Async Processing	Celery	5.4	Worker nodes that handle Wazuh API polling and email dispatching so the main API thread does not block.
Async Processing	Redis	7-alpine	In-memory message broker that queues the Celery tasks and temporarily stores the task results.
Data Persistence	PostgreSQL	18	Primary ACID-compliant relational database. Stores the normalized RBAC models and rule mappings.
Data Persistence	psycopg[binary]	3.2	The C-based adapter that allows the Django backend to communicate synchronously with PostgreSQL.

5.2 System Internal Components

The internal architecture of the ICMS platform consists of specialized modules that manage the compliance lifecycle. This modularity ensures a separation of concerns, with identity management, data governance, telemetry ingestion, and reporting handled by

independent units. Partitioning the system into these functional modules creates a framework that is resilient to external API failures. The following subsections detail the implementation logic of each system component.

5.2.1 Identity and Access Management (IAM) Module

The **IAM** module is the security foundation of the ICMS and enforces authentication and authorization policies. A custom identity model extends the core Django user system through a **UserProfile** entity to provide Role-Based Access Control (RBAC). The module recognizes three roles: Superadmins, who have full system access; Admins, who manage frameworks; and Auditors, who have read-only access to scan results and reports.

Authentication uses a stateless JSON Web Token (JWT) architecture to improve scalability. Upon credential verification, the system issues an access token and a rotating refresh token. For accounts requiring higher security, a two-factor authentication (2FA) layer is integrated using the Time-based One-Time Password (TOTP) algorithm. The IAM module uses a two-step login sequence where the user provides a 2FA code before the backend issues the JWT session. This policy is enforced at the API layer through Django REST Framework permissions and at the presentation layer using React route guards.

5.2.2 Governance and Compliance Data Engine

The **Governance and Compliance Data Engine** manages the relational mappings needed to translate regulatory requirements into technical controls. The engine stores compliance standards as **Framework** records. Each framework is divided into **Control** entities that represent specific security requirements. The engine links these controls to organizational **Policy** documents through a many-to-many relationship to document how each technical requirement is satisfied.

The engine includes mapping logic that connects security telemetry to regulatory clauses. A **WazuhMapping** system stores identifiers for Security Configuration Assessment (SCA) rules and ties them to governance controls. This allows the system to categorize security events into the appropriate compliance category. The engine also implements a risk-scoring algorithm that uses weights and thresholds to calculate compliance percentages. This structure ensures that changes in the security state of a remote endpoint are reflected in the organizational compliance posture.

5.2.3 SIEM Telemetry and Synchronization Module

The **SIEM Telemetry and Synchronization Module** is responsible for the ingestion of security data from the Wazuh SIEM manager. Given the volume of security logs and the latency of external API communication, this module is implemented as an asynchronous system. A Celery worker tier handles the execution of synchronization tasks, using Redis as a message broker to queue requests.

The synchronization lifecycle begins with a scheduled polling event managed by the Celery Beat service. The module authenticates with the Wazuh REST API and enumerates all active security agents. For each agent, the system retrieves SCA scan results as JSON payloads. The module stores the raw logs in a `JSONField` and extracts the pass/fail outcomes. This approach preserves audit evidence while providing data for dashboard rendering. The module implements a retry policy to handle network timeouts or SIEM downtime.

5.2.4 Automated Audit and Reporting Module

The **Automated Audit and Reporting Module** transforms scan results into audit artifacts. When a report is requested, the module performs an aggregation query across historical scan data, calculates compliance percentages, and identifies gaps where controls have failed. This aggregation is optimized with database-level annotations.

The module creates audit-ready evidence that meets regulatory requirements. The system uses a server-side rendering engine to convert HTML templates into PDF files. These documents include framework summaries, endpoint status lists, and gap analysis sections. Offloading PDF generation to the background worker tier ensures the user interface remains responsive. Files are served through a secure interface, removing the need for temporary storage of sensitive data on the local file system.

5.2.5 Frontend State and Visualization Layer

The **Frontend State and Visualization Layer** provides the interactive interface for managing the compliance lifecycle. Built as a React Single Page Application (SPA), this layer uses a component-based architecture. A global `AuthContext` provider manages the client-side session, JWT token persistence, and automatic token refresh logic through Axios interceptors. This ensures that outbound API requests are authenticated and that

the user’s session remains active.

Data visualization is a focus of this layer, providing insights into security states. A specialized charting library is used to render compliance score gauges, trend lines, and status distributions. These visualizations are driven by data from the backend. The frontend implements component mounting based on the user’s RBAC role. An auditor session initializes a different set of navigation and dashboard widgets than an administrator session, ensuring that users interact with relevant features.

5.3 Tools and Technologies

The implementation of the ICMS platform required a technology stack spanning from core backend infrastructure to AI-assisted development environments. Table 5.2 summarizes the tools and frameworks used throughout the project lifecycle.

Table 5.2: Summary of Tools, Technologies, and Development Stack

Category	Tool / Technology	Purpose in Project
Programming Languages	Python 3.12, JavaScript	Core backend API logic and frontend UI interactivity.
Backend Frameworks	Django 5.1, DRF	Database ORM mapping, migrations, and RESTful API creation.
Frontend Frameworks	React 19, Tailwind CSS 4.0	Component-based SPA architecture and utility-first UI styling.
Database & Messaging	PostgreSQL 18, Redis 7	ACID-compliant data persistence and asynchronous task queuing.
UI/UX Design	Google Stitch	Iterative wireframing, prototyping, and user interface design.
Development IDE	Antigravity	Primary orchestration platform and integrated coding environment.
AI Code Assistance	Claude (Opus 4.6, Sonnet 4.6), Gemini (3.1 Pro High, 3 Flash)	Advanced algorithmic modeling, automated code generation, and debugging support.
Documentation	LaTeX, MiKTeX, TikZ	Academic report typesetting and mathematical/architectural diagramming.

The selection of these technologies was driven by the requirements of a security governance platform. The Python/Django stack was chosen for the backend due to its handling of complex relational data and its support for authentication. On the presentation layer, the

React library was used to build a responsive Single Page Application (SPA). This choice allowed the system to manage UI states, such as dashboard updates and compliance gauges, without requiring full page reloads.

Prior to implementation, the system underwent an iterative UI/UX design process using Google Stitch. This platform was used for rapid prototyping, developing wireframes and mockups to validate user flows before writing frontend code. This phase established the aesthetic and ensured that navigation for security assessments was intuitive.

The development workflow used an AI-driven approach centered around the Antigravity integrated development environment. Antigravity was the orchestration hub, coordinating codebase management with AI modeling. Throughout the project, models such as Claude and Gemini were used for pair-programming tasks, specifically for the Wazuh telemetry parser and the Celery task retry logic. This environment enabled the debugging of container networking issues and the generation of API documentation.

Finally, the academic documentation was managed through LaTeX, MiKTeX, and TikZ. These tools ensured that the report adhered to academic typesetting standards and that architectural diagrams were precise. The TikZ library allowed for the programmatic generation of system topology diagrams.

Chapter 6

System Testing and Evaluation

The testing and evaluation phase verified that the Intelligent Compliance Management System (ICMS) meets the functional requirements and architectural specifications defined in the design phase. The process ensures that the platform provides security governance, accurate telemetry ingestion, and a stable user experience. Assessment methodologies included Graphical User Interface (GUI) validation, Usability assessments, Performance benchmarking, Exception Handling verification, and Security testing. These tests identified edge cases and validated the deterministic compliance mapping engine.

6.1 Critical Appraisal and Limitations

The ICMS platform demonstrates technical functionality in mapping heterogeneous Security Configuration Assessment (SCA) data to structured GRC controls. The asynchronous processing tier, using Celery and Redis, decouples SIEM polling from the user interface to maintain responsiveness during data ingestion. However, an engineering appraisal reveals limitations in the architecture. Periodic polling of the Wazuh REST API introduces latency between endpoint configuration changes and their reflection on the dashboard. This is suitable for periodic auditing but does not provide the near-real-time synchronization possible with a socket-based streaming architecture. Additionally, generating multi-page PDF audit reports is CPU-intensive; while managed by background workers, it increases the load on the containerized environment during reporting periods.

6.2 Testing Environment and Methodology

The Quality Assurance (QA) process provided full-stack coverage of the ICMS platform. Muhammad Hammad Chaghtai validated the frontend and integration layers, testing the GUI and usability of the React SPA. This included verifying route guarding, state persistence in the `AuthContext`, theme toggling, and the performance of Recharts-based visualizations. Because the Wazuh SIEM manager was hosted on localized hardware,

Muhammad Ali Bukhari validated the backend telemetry modules. This process included testing the Python client's authentication with the SIEM node, verifying network exception handling, and ensuring the integrity of API synchronization tasks. Testing occurred in a distributed containerized environment that simulated standard corporate home-lab conditions.

6.3 Functional System Test Cases

The following fourteen test cases constitute the QA suite used to validate the ICMS platform across IAM, frontend UI, backend API, SIEM telemetry, and reporting layers. Positive and negative scenarios are documented to demonstrate system correctness and exception handling.

Table 6.1: TC-01: Valid Superadmin Authentication (Positive)

Module Name	Identity and Access Management
Test Case ID	TC-01
Use Case ID	UC-01
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that a Superadmin user can successfully authenticate against the ICMS platform using valid credentials and is navigated to the correct role-specific dashboard upon token issuance.
Pre-Requisites	1. A Superadmin account exists in the PostgreSQL <code>UserProfile</code> table. 2. The Django backend API is active on port 8000.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin enters correct email and password credentials on the login screen.
Test Steps	1. Navigate to http://localhost:5173/login. 2. Enter credentials: <code>superadmin@icms.local</code> and the correct password. 3. Click "Login" and observe routing behavior and <code>localStorage</code> state.
Test Inputs	Email: <code>superadmin@icms.local</code> , Password: [valid credential]
Expected Results	The backend should issue a JWT access token and refresh token. The <code>AuthContext</code> should store these tokens, and the React Router should redirect to <code>/superadmin-dashboard</code> .
Actual Results	HTTP 200 received from <code>/api/auth/login/</code> ; JWT stored in <code>localStorage</code> under <code>grc_access_token</code> ; React Router pushed to <code>/superadmin-dashboard</code> in 120ms.
Status	PASS
Comments	Token issuance and client-side routing were completed well within acceptable latency thresholds for a LAN-connected environment.

Table 6.2: TC-02: Invalid Credential Rejection (Negative)

Module Name	Identity and Access Management
Test Case ID	TC-02
Use Case ID	UC-01
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the authentication endpoint rejects login attempts containing an incorrect password without issuing any session tokens, and that the UI communicates this failure visually to the user.
Pre-Requisites	1. A valid user account exists in the database. 2. The login form is rendered and interactive.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A user deliberately enters a wrong password to confirm the system does not grant unauthorized access.
Test Steps	1. Navigate to http://localhost:5173/login. 2. Enter <code>superadmin@icms.local</code> with an intentionally incorrect password. 3. Click "Login" and observe the HTTP response and UI feedback.
Test Inputs	Email: <code>superadmin@icms.local</code> , Password: <code>wrongpassword123</code>
Expected Results	The backend should return HTTP 401 Unauthorized. No JWT should be stored in <code>localStorage</code> . The UI should display a visible error notification.
Actual Results	HTTP 401 received from Django; no tokens stored; the <code>Toast</code> notification component rendered a red alert: "Invalid credentials. Please try again." within 85ms.
Status	PASS
Comments	The rejection path is handled entirely on the backend; no client-side credential caching occurred, confirming the security of the session management logic.

Table 6.3: TC-03: Global Theme Toggling — Light/Dark Mode (Positive)

Module Name	Frontend State and Visualization Layer
Test Case ID	TC-03
Use Case ID	UC-04
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the global <code>ThemeContext</code> provider correctly propagates a theme state change across all mounted React components, switching the UI between Light and Dark palettes without requiring a page reload.
Pre-Requisites	1. User is authenticated and viewing the main dashboard. 2. The theme toggle control is visible in the navigation bar.
Environment	React SPA (Windows Client, Chrome 134)
Test Scenario	A logged-in administrator clicks the theme toggle button while on the dashboard.
Test Steps	1. Log in and confirm the default Light Mode is active. 2. Click the sun/moon icon toggle in the top navigation bar. 3. Inspect the DOM's root element for the applied CSS class and verify all component colors update.
Test Inputs	Initial Theme: <code>light</code> , Toggle Action: <code>onClick</code>
Expected Results	The <code>ThemeContext</code> state should update from <code>light</code> to <code>dark</code> . All components subscribing to the context should re-render with the Dark Mode CSS variable palette, with no full-page reload.
Actual Results	<code>ThemeContext</code> state updated immediately; the <code>data-theme="dark"</code> attribute applied to the root <code><div></code> ; all sidebar, card, and table components re-rendered with the dark palette in under 30ms.
Status	PASS
Comments	The context-based theme architecture successfully avoids prop drilling and ensures consistent theming across all 20+ React components on the dashboard.

Table 6.4: TC-04: UI RBAC View Segregation (Positive)

Module Name	Identity and Access Management
Test Case ID	TC-04
Use Case ID	UC-05
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the React frontend enforces role-based component rendering by conditionally omitting privileged navigation items from the DOM when an Auditor-level session is active.
Pre-Requisites	1. Two accounts are available: one Auditor, one Superadmin. 2. Browser DevTools are open to inspect the DOM.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	An Auditor logs in and inspects the sidebar navigation to confirm restricted items are absent from the DOM.
Test Steps	1. Authenticate as an Auditor. 2. Observe the sidebar navigation menu after the dashboard loads. 3. Use Chrome DevTools to verify the "Manage Admins" and "Mapping Configuration" elements are not rendered in the DOM.
Test Inputs	Role: auditor, JWT Role Claim: {"role": "auditor"}
Expected Results	The Sidebar component should conditionally omit the "Manage Admins" and "Mapping Configuration" navigation items. These routes should also redirect to a 403 page if accessed directly.
Actual Results	Sidebar rendered 4 navigation items for Auditor vs. 9 for Superadmin. DevTools confirmed the privileged <NavItem> components were not mounted in the DOM, not merely hidden with CSS.
Status	PASS
Comments	DOM-level omission (as opposed to CSS display:none) is the correct security pattern as it prevents advanced users from unhiding elements via browser console commands.

Table 6.5: TC-05: Unauthorized API Write Attempt (Negative)

Module Name	Identity and Access Management
Test Case ID	TC-05
Use Case ID	UC-05
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Verifying that the Django REST Framework permission layer blocks write operations originating from an Auditor-level JWT, even when the HTTP request is constructed manually to bypass the UI restrictions.
Pre-Requisites	1. User is authenticated as an Auditor. 2. Postman or curl is used to craft a direct API request.
Environment	Postman (Windows Client), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	An Auditor uses Postman to construct a direct POST request to the framework management endpoint, attempting to create a new compliance control mapping.
Test Steps	1. Authenticate as Auditor via <code>/api/auth/login/</code> to receive a restricted JWT. 2. Attach the JWT and construct a POST request to <code>/api/frameworks/</code>. 3. Submit the request and record the HTTP response code and body.
Test Inputs	Role: <code>auditor</code> , Endpoint: <code>POST /api/frameworks/</code> , Payload: <code>{"name": "Unauthorized_Frame"}</code>
Expected Results	DRF <code>IsAdminOrSuperAdmin</code> permission class should intercept the request and return HTTP 403 Forbidden. No database record should be created.
Actual Results	HTTP 403 Forbidden returned in 52ms; response body: <code>{"detail": "You do not have permission to perform this action."}</code> ; zero database writes confirmed via query log.
Status	PASS
Comments	The permission layer operates at the view level, independent of the UI, ensuring API security even against direct HTTP attacks.

Table 6.6: TC-06: Deterministic Framework Mapping (Positive)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-06
Use Case ID	UC-03
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that a Superadmin can create a valid regulatory mapping linking a specific Wazuh SCA Rule ID to a named ISO 27001 control, and that the record is persisted correctly in the database.
Pre-Requisites	1. ISO 27001 Framework and Control A.12.1.2 are seeded in the database. 2. Wazuh Rule ID 19502 does not yet have a mapping.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin navigates to the mapping panel and creates a new rule-to-control entry.
Test Steps	1. Log in as Superadmin and navigate to "Mapping Configuration". 2. Input Wazuh Rule ID 19502, select target control A.12.1.2 (Change Management), and click "Save". 3. Verify the new row appears in the mapping table on the page.
Test Inputs	Wazuh Rule ID: 19502, Control: A.12.1.2 - Change Management, Framework: ISO 27001
Expected Results	A new WazuhMapping record should be persisted to the PostgreSQL database and reflected in the UI mapping table.
Actual Results	POST /api/mappings/ returned HTTP 201 Created; database commit confirmed in 110ms; the new mapping appeared in the UI table immediately after the Axios response resolved.
Status	PASS
Comments	The successful creation of this mapping is foundational to the entire automated scoring pipeline; the 110ms DB commit time is well within acceptable thresholds.

Table 6.7: TC-07: Duplicate Mapping Collision (Negative)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-07
Use Case ID	UC-03
Tester Name(s)	Muhammad Hammad Chaghtai
Test Description	Verifying that the database unique constraint on the WazuhMapping model prevents the creation of a duplicate entry for an already-mapped Wazuh Rule ID, and that the backend returns a structured validation error to the UI.
Pre-Requisites	1. Wazuh Rule ID 19502 is already mapped to Control A.12.1.2 (established in TC-06). 2. User is authenticated as Superadmin.
Environment	React SPA (Windows Client, Chrome 134), Django/PostgreSQL (Docker Ubuntu)
Test Scenario	A Superadmin attempts to create an identical mapping entry for the same Rule ID and Control combination.
Test Steps	1. Navigate to "Mapping Configuration". 2. Attempt to save a new mapping using the same Rule ID 19502 and the same Control A.12.1.2. 3. Observe the error response from the backend.
Test Inputs	Wazuh Rule ID: 19502, Control: A.12.1.2, Framework: ISO 27001 (duplicate)
Expected Results	PostgreSQL should raise an IntegrityError for the unique-together constraint. The DRF serializer should catch this and return HTTP 400 Bad Request. The UI should display a distinct error alert.
Actual Results	IntegrityError caught by the DRF serializer; HTTP 400 returned with body <code>{"non_field_errors": ["Duplicate Mapping Exception: This rule is already mapped."]}</code> ; UI displayed an orange warning toast in 95ms.
Status	PASS
Comments	The constraint is enforced at the database level, making it impossible to create duplicate mappings even via direct API calls.

Table 6.8: TC-08: Wazuh API Handshake and Token Retrieval (Positive)

Module Name	SIEM Telemetry and Synchronization
Test Case ID	TC-08
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the <code>WazuhAPIClient</code> service's capability to establish an authenticated session with the Wazuh Manager REST API by exchanging Basic Auth credentials for a short-lived Bearer token.
Pre-Requisites	1. Wazuh Manager is active at <code>192.168.100.100:55000</code>. 2. Environment variables <code>WAZUH_API_USER</code> and <code>WAZUH_API_PASSWORD</code> are configured.
Environment	Django Backend (Docker Ubuntu), Wazuh Manager 4.x (Physical Laptop)
Test Scenario	The ICMS backend initializes the <code>WazuhAPIClient</code> and calls the <code>authenticate()</code> method.
Test Steps	1. Trigger <code>WazuhAPIClient.authenticate()</code> from the Django shell. 2. Monitor outbound HTTPS traffic on Port 55000. 3. Confirm the Bearer token is stored on the client instance.
Test Inputs	Host: <code>192.168.100.100</code> , Port: <code>55000</code> , SSL Verify: <code>False</code>
Expected Results	The client should call <code>GET /security/user/authenticate</code> with Basic Auth and store the resulting JWT as <code>self._token</code> .
Actual Results	<code>InsecureRequestWarning</code> suppressed; HTTP 200 received; Bearer token retrieved and stored in 0.8s despite self-signed certificate overhead.
Status	PASS
Comments	Disabling SSL verification is an accepted trade-off in a home-lab deployment; production environments should use a valid CA-signed certificate.

Table 6.9: TC-09: Wazuh Connection Timeout Exception (Negative)

Module Name	SIEM Telemetry and Synchronization
Test Case ID	TC-09
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Simulating a complete network failure during SIEM synchronization to validate that the Celery worker catches the timeout exception and triggers an exponential backoff retry rather than crashing silently.
Pre-Requisites	1. Wazuh Manager taken offline (firewall rule blocks Port 55000). 2. Celery worker configured with <code>max_retries=3</code>.
Environment	Django Backend (Docker Ubuntu), Wazuh Manager 4.x (Offline)
Test Scenario	A scheduled telemetry synchronization task fires while the SIEM node is unreachable.
Test Steps	1. Block outbound traffic to Port 55000. 2. Execute <code>sync_wazuh_scans.delay()</code>. 3. Monitor Celery logs for exception handling and retry behavior.
Test Inputs	Target Host: 192.168.100.100:55000, Network State: Connection Refused
Expected Results	<code>requests.exceptions.Timeout</code> caught; Celery calls <code>self.retry(countdown=60)</code> to re-enqueue the task.
Actual Results	Timeout raised after 15s; Celery logged "Task retrying in 60s"; task completed successfully on second attempt after connectivity was restored.
Status	PASS (Handled Exception)
Comments	The retry mechanism prevents data loss during transient network failures; the 60-second countdown avoids flooding a recovering SIEM node.

Table 6.10: TC-10: Asynchronous Telemetry Polling via Celery Beat (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-10
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the full end-to-end execution of the <code>sync_wazuh_scans</code> Celery Beat periodic task, from queue dispatch to database commit, confirming the API thread is never blocked during the operation.
Pre-Requisites	1. Redis broker is active. 2. At least 5 agents registered on Wazuh Manager. 3. A periodic task entry exists in the <code>django_celery_beat</code> schedule table.
Environment	Celery Worker / Redis (Docker Ubuntu), Wazuh Manager 4.x
Test Scenario	The scheduled synchronization task is triggered by Celery Beat to poll all 5 registered endpoints.
Test Steps	1. Monitor Celery Beat logs for the dispatch event. 2. Confirm all 5 agents are polled sequentially. 3. Query the <code>ScanResult</code> table for new records after completion.
Test Inputs	Task: <code>sync_wazuh_scans</code> , Agent Count: 5, Trigger: <code>Manual</code>
Expected Results	All 5 endpoints polled; new <code>ScanResult</code> records committed; Django API process remains responsive throughout.
Actual Results	1,200 SCA check rows updated across 5 agents in 12.4s; simultaneous <code>GET /api/dashboard/</code> request responded in 78ms, confirming non-blocking operation.
Status	PASS
Comments	The 78ms concurrent API response confirms the Celery worker tier decoupling is functioning correctly under active load.

Table 6.11: TC-11: Unstructured JSON Log Ingestion (Positive)

Module Name	Governance and Compliance Data Engine
Test Case ID	TC-11
Use Case ID	UC-02
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Verifying the integrity of the raw log ingestion pipeline when saving a large, semi-structured Wazuh SCA JSON payload into the PostgreSQL <code>jsonb</code> field on the <code>ScanResult</code> model.
Pre-Requisites	1. Wazuh API returning a high-density SCA check payload. 2. PostgreSQL 18 database active and accepting connections.
Environment	Django Backend (Docker Ubuntu), PostgreSQL 18
Test Scenario	A synchronization task receives an 18KB JSON object from the Wazuh API and saves it to <code>raw_log_data</code> .
Test Steps	1. Execute the sync task for a high-density agent. 2. Query the <code>ScanResult</code> table for the <code>raw_log_data</code> field. 3. Compare character count and JSON key structure with the source SIEM payload.
Test Inputs	Payload Source: Wazuh SCA endpoint, Payload Size: 18KB, Field: <code>jsonb</code>
Expected Results	The entire JSON object stored intact with no truncation, and fully traversable via Django ORM for audit retrieval.
Actual Results	18KB payload committed to <code>raw_log_data</code> in 1.1s; character-count comparison confirmed zero data loss; JSON tree fully traversable via ORM.
Status	PASS
Comments	The dual-storage approach (structured boolean + raw JSON) preserves original evidence while enabling fast dashboard queries.

Table 6.12: TC-12: Dynamic Compliance Score Recalculation (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-12
Use Case ID	UC-04
Tester Name(s)	Muhammad Ali Bukhari
Test Description	Validating the scoring aggregation logic by confirming that a new scan result resolving a previously non-compliant control triggers a recalculation of the global compliance percentage, reflected immediately in the frontend gauge.
Pre-Requisites	1. Agent 004 has a baseline score of 75.0%. 2. A failed SCA check for a mapped control exists in the database.
Environment	Celery Worker (Docker Ubuntu), React SPA (Windows Client)
Test Scenario	A manual synchronization resolves a previously failing control, incrementing the global score.
Test Steps	1. Trigger <code>sync_wazuh_scans.delay(agent_id=4)</code> manually. 2. Query <code>DashboardSummaryView</code> for the updated compliance score. 3. Refresh the dashboard and observe the <code>ComplianceGauge</code> animation.
Test Inputs	Agent ID: 004, Control Weight: +6.2%, Initial Score: 75.0%
Expected Results	Global score increments from 75.0% to 81.2%; the React dashboard re-renders the <code>ComplianceGauge</code> SVG with the new value.
Actual Results	Score updated to 81.2% in the database within 2.4s of task completion; gauge SVG re-animated to the new value in 40ms without a page reload.
Status	PASS
Comments	The real-time gauge update is achieved through the Axios polling mechanism in the <code>Dashboard</code> component, re-fetching data every 30 seconds.

Table 6.13: TC-13: Comprehensive PDF Audit Report Generation (Positive)

Module Name	Automated Audit and Reporting Module
Test Case ID	TC-13
Use Case ID	UC-06
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Testing the server-side PDF rendering pipeline's ability to aggregate multi-agent compliance data, apply a Django HTML template, and produce a structurally sound binary PDF with correct page breaks and table alignment.
Pre-Requisites	1. Database contains at least 50 historical <code>ScanResult</code> records across 5 agents. 2. <code>xhtml2pdf</code> is installed and the report HTML template is configured.
Environment	Django Backend (Docker Ubuntu), React SPA (Windows Client)
Test Scenario	An Auditor clicks "Generate Full Report" for the ISO 27001 framework on the Reports page.
Test Steps	1. Authenticate as Auditor and navigate to the Reports page. 2. Select framework "ISO 27001" and click "Generate Report". 3. Download the returned PDF binary and inspect page breaks and table layout.
Test Inputs	Framework: ISO 27001, Date Range: Last 30 Days, Agent Count: 5
Expected Results	A multi-page PDF containing a framework summary, per-agent compliance tables, and a gap analysis section, with clean page breaks and no overlapping elements.
Actual Results	15-page PDF document generated in 4.2s; all inter-section page breaks executed correctly without table overlap; all data values matched the live database query.
Status	PASS
Comments	The 4.2s generation time for a 15-page dossier is acceptable; offloading to the worker tier ensured the auditor's browser session remained interactive throughout.

Table 6.14: TC-14: Asynchronous SMTP Alert Dispatch (Positive)

Module Name	Asynchronous Processing Engine
Test Case ID	TC-14
Use Case ID	UC-05
Tester Name(s)	Muhammad Hammad Chaghtai & Muhammad Ali Bukhari
Test Description	Validating the automated alert system’s capability to detect a compliance score threshold breach and dispatch a structured HTML email notification via the SMTP relay, without blocking the primary ingestion pipeline.
Pre-Requisites	1. System settings configured with an alert threshold of 80%. 2. SMTP credentials pointing to a MailTrap test inbox are active.
Environment	Celery Worker (Docker Ubuntu), SMTP Relay (MailTrap)
Test Scenario	A telemetry synchronization ingests data dropping Agent 004’s compliance score below the 80% critical threshold, triggering the alert pipeline.
Test Steps	1. Insert a simulated scan result lowering Agent 004’s score to 65.0%. 2. Monitor Celery logs for the <code>send_compliance_alert</code> task dispatch. 3. Check the MailTrap inbox for the received email.
Test Inputs	Agent ID: 004, New Score: 65.0%, Configured Threshold: 80.0%
Expected Results	The scoring module detects the breach, enqueues a <code>send_compliance_alert</code> task, and the worker dispatches an HTML email containing the agent name, current score, and framework details.
Actual Results	Threshold breach detected post-aggregation; <code>send_compliance_alert</code> task dispatched in 1.1s; HTML email received in MailTrap containing Agent 004 metadata, score 65.0%, and a direct link to the agent detail page.
Status	PASS
Comments	The dedicated SMTP worker ensures email dispatch latency does not delay subsequent compliance ingestion tasks queued behind it.

6.4 Non-Functional System Testing

The evaluation of the ICMS platform extended beyond functional correctness to assess the system’s operational constraints, scalability, and overall user experience. This phase of non-functional testing focused on ensuring that the architectural decisions—such as

the separation of concerns and the use of an asynchronous task queue—translate into a performant, secure, and usable application for security analysts.

6.4.1 Performance and Load Testing

The performance evaluation confirmed that the ICMS platform maintains high responsiveness under typical operational loads. Benchmarking revealed that standard API requests, such as retrieving framework lists or mapping configurations, consistently resolve in under 150ms. To handle more intensive I/O operations, such as polling large datasets from the Wazuh SIEM or rendering multi-page PDF reports, the system utilized a Celery worker tier. This architectural choice successfully offloads long-running processes from the main request-response cycle, preventing the Nginx and Django gateway from timing out during heavy stress. Even when processing 50 simultaneous synchronization tasks, the system remained stable, completing all operations without degrading the user interface performance.

Table 6.15: Performance Test Results

Test Scenario	Metric Measured	Result	Status
React SPA cold start to Login	Screen load time	Under 1.5 seconds	Pass
Django API framework list fetch	Response latency	Under 150 ms	Pass
Wazuh SIEM Sync (50 agents)	Celery execution time	Under 15 seconds	Pass
Compliance gauge re-render	UI update latency	Under 50 ms	Pass
ISO 27001 PDF generation (15 pages)	Server rendering time	Under 4.5 seconds	Pass
SMTP alert email dispatch	Queue delivery time	Under 1.5 seconds	Pass

6.4.2 Security and Penetration Testing

A defense-in-depth strategy was implemented to secure the platform against unauthorized access and data manipulation. The system relies on the Django ORM to provide inherent protection against SQL injection attacks by utilizing parameterized queries for all database interactions. Identity management is handled through a stateless JWT ar-

chitecture, utilizing short-lived access tokens and rotating refresh tokens to minimize the window for potential session hijacking. Additionally, the integration of TOTP-based 2FA provides a cryptographically secure second layer of authentication, ensuring that administrative access remains protected even in the event of primary credential compromise.

Table 6.16: Security Test Results

Security Control	Test Description	Status
JWT Session Enforcement	Attempted access to protected Dashboard without token; React Router successfully redirected to login.	Pass
Role-Based Access Control	Attempted to POST mapping rules using an Auditor token; DRF returned HTTP 403 Forbidden.	Pass
SQL Injection Prevention	Submitted malicious SQL payloads in search fields; Django ORM parameterized queries safely.	Pass
TOTP 2FA Verification	Attempted to bypass the 2FA prompt after password entry; backend withheld the final access token.	Pass
XSS Sanitization	Injected HTML scripts into framework descriptions; React DOM safely escaped the output.	Pass

6.4.3 Usability and Accessibility Testing

The user interface is designed to manage cognitive load for analysts in high-pressure Security Operations Center (SOC) environments. The implementation includes a global `ThemeContext` that allows toggling between Light and Dark modes. This gives analysts control over the UI palette based on local lighting conditions to reduce eye strain. The platform organizes compliance data into paginated and filterable tables to ensure that datasets remain manageable without overwhelming the interface.

Table 6.17: Usability Test Task Completion Results

Task	Avg. Completion Time	Success Rate	Status
Log in and complete 2FA verification	0.8 minutes	5 / 5 (100%)	Pass
Toggle Light/Dark mode context	0.2 minutes	5 / 5 (100%)	Pass
Map a Wazuh Rule to an ISO Control	1.5 minutes	5 / 5 (100%)	Pass
Trigger a manual endpoint scan	1.0 minutes	5 / 5 (100%)	Pass
Generate and download PDF report	1.2 minutes	4 / 5 (80%)	Pass

6.4.4 Compatibility and Deployment Testing

The ICMS platform underwent compatibility testing to ensure a consistent experience across modern web browsers, including Chrome, Firefox, and Edge. The React SPA maintained its layout integrity and functionality across these environments, confirming the reliability of the frontend codebase. Deployment testing validated the containerized architecture by performing multiple teardowns and spin-ups using Docker Compose. By encapsulating the entire stack—including Django, React, Redis, Celery, and PostgreSQL—the deployment remains deterministic across different host environments. This containerized approach ensures that the platform can be reliably deployed on any Ubuntu-based machine without encountering dependency conflicts.

Table 6.18: Compatibility and Deployment Test Results

Environment / Scenario	Test Description	Status
Cross-Browser UI Rendering	Executed React SPA on Chrome, Firefox, and Edge; Flexbox layouts and charts rendered correctly.	Pass
Docker Compose Teardown	Executed <code>docker-compose down -v</code> and rebuilt stack; all 7 containers initialized with Exit Code 0.	Pass
Database Volume Persistence	Restarted PostgreSQL container; verified that previous user accounts and compliance data remained intact.	Pass
Celery Broker Recovery	Temporarily stopped Redis container; verified Celery gracefully resumed task queuing upon Redis restart.	Pass

Chapter 7

Conclusions

The Intelligent Compliance Management System (ICMS) demonstrates that integrating Security Information and Event Management (SIEM) telemetry into a Governance, Risk, and Compliance (GRC) engine automates the auditing lifecycle. The platform connects regulatory frameworks to endpoint security through a mapping algorithm. The implementation shows that translating Wazuh Security Configuration Assessment (SCA) alerts into ISO 27001 controls allows an organization to maintain a continuous compliance posture.

The implementation results in several engineering conclusions. A hybrid database schema, using PostgreSQL's `JSONField` for Wazuh logs alongside relational tables for user roles and frameworks, provides a mechanism for ingesting SIEM data while maintaining ACID compliance. The evaluation phase confirmed that offloading I/O-bound operations, such as API polling and PDF generation, to a Celery worker queue prevents API gateway timeouts. The deployment also shows that stateless JSON Web Tokens (JWT) combined with a React Context provider enforces Role-Based Access Control (RBAC) without server-side session management overhead.

Future development could expand this project into an enterprise product. Technical enhancements include transitioning SIEM integration from REST API polling to a WebSocket or Webhook-driven architecture to reduce latency and enable real-time compliance recalculations. The containerized topology could also be upgraded from Docker Compose to Kubernetes to support horizontal scaling. The long-term objective for the ICMS platform is to refine the orchestration engine for release as an open-source solution. Open-sourcing the codebase would support community development and allow for additional integration modules for SIEM providers like Splunk or Elastic Security, while expanding the library of supported regulatory frameworks.

Appendix A

User Manual

A.1 Purpose

The Intelligent Compliance Management System (ICMS) is a Governance, Risk, and Compliance (GRC) platform. It automates the auditing process by mapping technical telemetry from the Wazuh SIEM to regulatory frameworks, reducing the need for manual spreadsheet-based assessments.

A.2 System Requirements

The following requirements are for system operation:

- **Device:** Workstation with a minimum 1080p resolution recommended for viewing data tables.
- **Web Browser:** Google Chrome v100+, Mozilla Firefox v100+, or Microsoft Edge.
- **Network:** Intranet or VPN access to the ICMS environment.
- **Authentication:** Authenticator Application (e.g., Google Authenticator, Authy) for TOTP Two-Factor Authentication.

A.3 System Access

- Open a web browser and navigate to the ICMS URL.
- Enter the registered email and password.
- Enter the 6-digit TOTP code from the authenticator application.

- The system directs the user to the Dashboard based on the assigned role (Super-admin, Admin, or Auditor).

A.4 Main User Steps

Step 1 – Authentication and Role Verification

- Users authenticate through the login portal.
- The system verifies the session and displays the interface corresponding to the user privilege level.

Step 2 – Dashboard Overview

- Use the Home Dashboard to view the organizational compliance posture.
- View the compliance percentages for active frameworks.
- Use the theme switcher in the navigation bar to toggle between Light and Dark modes.

Step 3 – Framework and Control Management (Admins)

- Use the "Frameworks" tab to view regulatory standards.
- Select a framework to view its clauses and technical controls.

Step 4 – Telemetry Mapping Configuration (Superadmins)

- Use the "Mapping Configuration" portal.
- Link Wazuh Security Configuration Assessment (SCA) Rule IDs to GRC controls.
- Save the mapping to categorize incoming SIEM events.

Step 5 – Wazuh Telemetry Synchronization

- The system polls the Wazuh manager in the background.

- Administrators can click "Trigger Manual Sync" on the Scans page for an immediate update.
- View synchronization logs to verify data ingestion.

Step 6 – Endpoint Monitoring

- Use the "Endpoints" view to monitor specific systems.
- Select a workstation or server to view its compliance score and failed security checks.

Step 7 – Automated Audit Report Generation

- Use the "Reports" module to generate documentation.
- Select the regulatory framework and date range.
- Click "Generate PDF" to download the audit report.

Step 8 – System and User Management

- Superadmins use "Manage Users" to provision accounts and assign roles.
- Configure SMTP alerts in System Settings for email notifications upon compliance failure.

A.5 Logout

- Click the user profile icon in the navigation shell.
- Select "Logout" to end the session and clear browser storage.

A.6 Support

- For credential recovery or 2FA resets, contact the IT Security Administrator.
- For Wazuh agent connectivity issues, contact the SIEM infrastructure team.

A.7 Note

The ICMS platform automates compliance visibility and reporting. The system provides mappings based on SIEM telemetry and functions as an orchestration and governance tool. Maintaining compliance requires remediation of identified vulnerabilities. The ICMS highlights security gaps, but administrative action on the endpoints is necessary to resolve them.

References

- [1] T. T. Bukhari, O. Oladimeji, E. D. Etim, and J. O. Ajayi, “Automated control monitoring: A new standard for continuous audit readiness,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 7, no. 3, pp. 711–735, 2021. Cited on p. [1](#).
- [2] Joint Task Force, “Security and privacy controls for information systems and organizations,” Tech. Rep. NIST Special Publication 800-53 Revision 5, National Institute of Standards and Technology, 2020. Cited on pp. [1](#), [8](#), and [9](#).
- [3] G. Quispe-Kobashikawa, C. Zuloaga-Estrada, P. S. Castañeda Vargas, and A. D. García-Núñez, “Enhancing information security in technology small and medium-sized enterprises: A metrics-driven model based on ISO/IEC 27001:2022,” *IEEE Access*, 2026. Cited on pp. [1](#) and [8](#).
- [4] L. Yang, Z. Chen, C. Wang, Z. Zhang, S. Booma, P. Cao, C. Adam, A. Withers, Z. Kalbarczyk, R. K. Iyer, and G. Wang, “True attacks, attack attempts, or benign triggers? an empirical measurement of network alerts in a security operations center,” in *Proceedings of the 33rd USENIX Security Symposium*, (Philadelphia, PA, USA), 2024. Cited on pp. [2](#) and [11](#).
- [5] P. Best, “Auditing during a pandemic: Can continuous controls monitoring (ccm) address challenges facing internal audit departments?,” *Pacific Accounting Review*, vol. 35, no. 5, pp. 727–745, 2023. Cited on p. [8](#).
- [6] M. A. Jamali and A. Masood, “Cybersecurity on a budget: Evaluating security and performance of open-source SIEM solutions for SMEs,” *PLOS ONE*, vol. 19, no. 3, p. e0301183, 2024. Cited on p. [9](#).
- [7] D. Poudel, “A unified automation framework for CIS benchmark compliance and real-time remediation on ubuntu using wazuh, ansible and FastAPI,” *International Journal of Innovative Science and Research Technology*, vol. 11, no. 4, 2026. Cited on p. [9](#).
- [8] R. F. Aji and N. P. Utami, “Evaluating field flexibility approaches in relational databases: A performance study of JSON and column-oriented models in library systems,” *International Journal of Advanced Computer Science and Applications*, vol. 17, no. 1, 2026. Cited on pp. [9](#) and [58](#).

- [9] H. D. Cat, “Implementing security measures in web chat applications using django framework,” bachelor’s thesis, Tampere University, 2025. Cited on p. [10](#).
- [10] S. Ahmad, M. Arif, J. Ahmad, and S. Mehfuz, “A TOTP-based secure data storage system in the cloud environment using the JWT token approach,” *International Journal of System Assurance Engineering and Management*, vol. 16, no. 4, pp. 1565–1578, 2025. Cited on p. [10](#).
- [11] M. Amer and T. S. Sobh, “Biometric JSON web tokens (BJWT): Enhancing web API security with biometric key exchange and OTP-JWT authentication,” *International Journal of Computer Applications*, vol. 186, no. 39, pp. 15–21, 2024. Cited on p. [11](#).
- [12] E. Casalicchio and S. Iannucci, “The state-of-the-art in container technologies: Application, orchestration and security,” *Concurrency and Computation: Practice and Experience*, vol. 32, no. 17, p. e5668, 2020. Cited on pp. [11](#) and [57](#).
- [13] V. S. Thokala, “Utilizing docker containers for reproducible builds and scalable web application deployments,” *Journal of Information Systems Engineering and Management*, vol. 10, no. 45s, p. 725, 2025. Cited on p. [11](#).
- [14] J. Happa, I. Agraftotis, M. Helmhout, T. Bashford-Rogers, M. Goldsmith, and S. Creese, “Assessing a decision support tool for SOC analysts,” *Digital Threats: Research and Practice*, 2020. Cited on p. [11](#).
- [15] S. A. Chamkar, M. Zaydi, Y. Maleh, and N. Gherabi, “Improving threat detection in wazuh using machine learning techniques,” *Journal of Cybersecurity and Privacy*, vol. 5, no. 2, p. 34, 2025. Cited on p. [56](#).